Foundations of Spatial AI

04 – Stereo Depth Estimation

Detailed lecture notes

Notes based on the slides by L. Di Giammarino and E. Giacomini
(Sapienza University of Rome)

October 5, 2026

Abstract

These notes follow the slide deck *04 Stereo Depth Estimation* (60 slides) section by section; every header carries its slide range. Formal statements (definitions, lemmas, proofs) are merged in where the slides introduce a topic, and blue *Intuition* boxes explain the "why" in plain words. **Source note:** only the slide deck was uploaded, no companion chapter. The formal content therefore comes from standard references that the slides themselves cite (Hartley–Zisserman / Szeliski for geometry) and from the papers named on the slides (Zbontar–LeCun, Mayer et al., Kendall et al., Wang et al., Leroy et al.). Where a statement goes beyond what the slides say, it is marked as such. Diagrams are redrawn in TikZ; photographs are left as dashed placeholders that name the slide to copy from.

Contents

Agenda and Learning Objectives (slides 1–2) **2**

4.1 Two-View Geometry *(slides 3–13)* **2**
4.1.1 Depth Cues and Binocular Stereopsis *(slides 4)* . . . 2
4.1.2 Two-View Stereo Matching *(slides 5)* . . . 2
4.1.3 The Classical Stereo Pipeline *(slides 6)* . . . 3
4.1.4 Epipolar Geometry *(slides 7)* . . . 3
4.1.5 Image Rectification *(slides 8)* . . . 4
4.1.6 Rectified Epipolar Constraint *(slides 9)* . . . 5
4.1.7 Calculating the Rectifying Rotation Matrix *(slides 10)* . . . 5
4.1.8 Rectified Pair and Disparity *(slides 11)* . . . 6
4.1.9 Disparity to Depth *(slides 12)* . . . 6
4.1.10 Worked Example: Baseline and Depth Error *(slides 13)* . . . 7

4.2 Block Matching *(slides 14–21)* **8**
4.2.1 Correspondence Ambiguity *(slides 15)* . . . 8
4.2.2 Similarity Metrics *(slides 16)* . . . 8
4.2.3 Block Matching *(slides 17)* . . . 9
4.2.4 Block Matching: Half Occlusions *(slides 18)* . . . 10
4.2.5 Block Matching: Assumption Violations *(slides 19)* . . . 11
4.2.6 Effect of Window Size *(slides 20)* . . . 12
4.2.7 Left-Right Consistency Test *(slides 21)* . . . 12

4.3 Siamese Networks *(slides 22–28)* **13**
4.3.1 Siamese Networks for Stereo Matching *(slides 23)* . . . 13
4.3.2 Siamese Network Architecture *(slides 24)* . . . 14
4.3.3 Training Triplets *(slides 25)* . . . 15
4.3.4 Hinge Loss *(slides 26)* . . . 16
4.3.5 Comparisons *(slides 27)* . . . 17
4.3.6 Similarity Constraint: Failure Cases *(slides 28)* . . . 17

4.4 End-to-End Learning *(slides 29–31)* **18**
4.4.1 DispNet and Synthetic Data *(slides 30)* . . . 18
4.4.2 GC-Net *(slides 31)* . . . 19

4.5 Stereo Without Calibration *(slides 32–45)* **20**
4.5.1 The Cost of Calibration *(slides 33)* . . . 20
4.5.2 Limits of Rectification and Local Matching *(slides 34)* . . . 21
4.5.3 Pointmaps *(slides 35)* . . . 21
4.5.4 The DUSt3R Architecture *(slides 36)* . . . 22
4.5.5 MASt3R Network Output *(slides 37)* . . . 23
4.5.6 Training Objective and Metric Scale *(slides 38)* . . . 23
4.5.7 The Matching Head *(slides 39)* . . . 24
4.5.8 InfoNCE and the Hinge Loss *(slides 40)* . . . 24
4.5.9 Effect of 3D Grounding *(slides 41)* . . . 25
4.5.10 Reciprocal Matching *(slides 42)* . . . 25
4.5.11 MASt3R Training Details *(slides 43)* . . . 26
4.5.12 Results *(slides 44)* . . . 26
4.5.13 The Pipeline Revisited *(slides 45)* . . . 27

Summary **28**
Comparison of the methods covered . . . 28
Key take-aways . . . 28

Agenda and Learning Objectives *(slides 1–2)*

The lecture has five parts: 4.1 Two-View Geometry, 4.2 Block Matching, 4.3 Siamese Networks, 4.4 End-to-End Learning, 4.5 Stereo Without Calibration. After it you should be able to:

1. explain why calibration and rectification reduce stereo matching to a 1D search, and derive $z = fb/d$ together with its error;
2. explain block matching, its failure modes and the left-right consistency test;
3. describe how learned matching costs and end-to-end networks entered stereo;
4. explain how matching in 3D removes calibration and rectification from the matching step, and what it gives up.

4.1 Two-View Geometry *(slides 3–13)*

4.1.1 Depth Cues and Binocular Stereopsis *(slides 4)*

Recovering depth from a single image is *ill-posed* (previous lecture). Three ways around it:

- **Time-of-flight (ToF)** sensors measure the round-trip time to an obstacle ⇒ metric depth.
- **Stereo vision** triangulates using two views; because the baseline is known, depth is metric.
- **A depth prior** (object assumptions or a neural network) usually gives depth only *up to an affine* transformation.

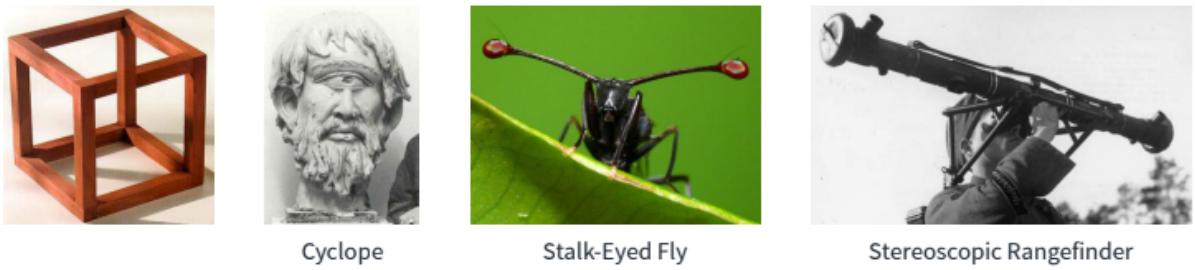


Figure 1: Historical and natural binocular systems: Cyclops (cube illusion), stalk-eyed fly, stereoscopic rangefinder

Intuition

Your two eyes are a metric ruler *because* the distance between them is fixed. A single eye (or a monocular network) can say "this is nearer than that" but not "this is 12 m away". Stereo trades a second camera for a physical scale.

4.1.2 Two-View Stereo Matching *(slides 5)*

Goal: recover the *disparity* (proportional to inverse depth) for every pixel of a reference image from two input images.

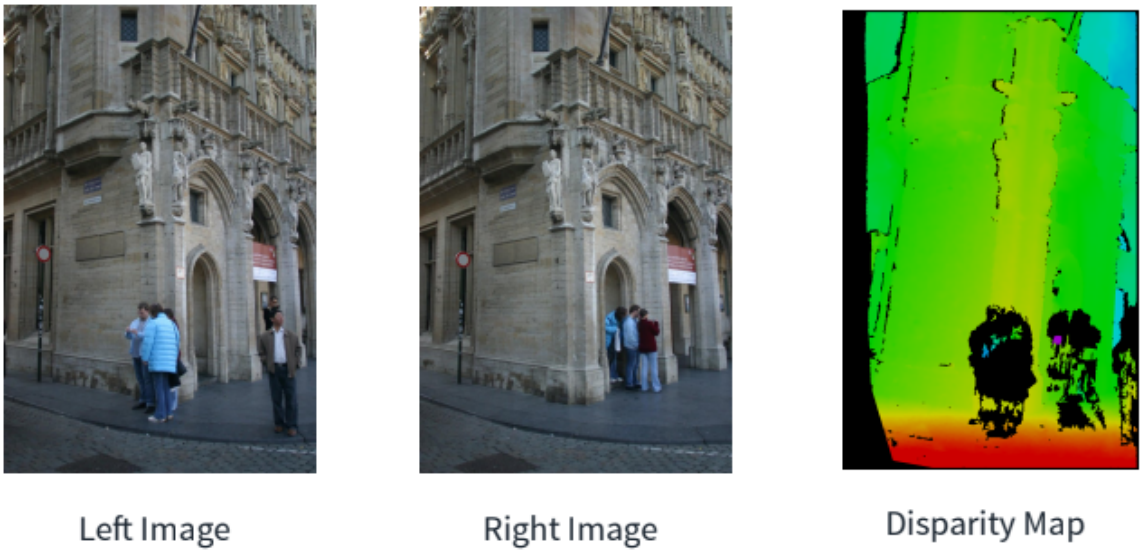


Figure 2: Left image, right image and the colour-coded disparity map (Geiger et al., ACCV 2010).

4.1.3 The Classical Stereo Pipeline *(slides 6)*

Task: build a dense 3D model of a static scene from two images. Pipeline:

1. calibrate the cameras intrinsically and extrinsically;
2. rectify the images using the calibration;
3. compute a disparity map for the reference image;
4. remove outliers with a consistency/occlusion test;
5. obtain depth from disparity using the calibration;
6. build a 3D model, e.g. by volumetric fusion and meshing.

Steps 1–2 (highlighted on the slide) are exactly what Section 4.5 will remove; we revisit this list at the end (*The Pipeline Revisited*).

4.1.4 Epipolar Geometry *(slides 7)*

Definition 4.1.1 (Essential matrix)**.** Let a 3D point $\boldsymbol{x}$ project to $\bar{\boldsymbol{x}}_1 \propto \boldsymbol{K}_1\boldsymbol{x}$ in camera 1 and to $\bar{\boldsymbol{x}}_2 \propto \boldsymbol{K}_2(\boldsymbol{R}\boldsymbol{x}+\boldsymbol{t})$ in camera 2 ($\bar{\boldsymbol{x}}$ are homogeneous pixel coordinates). The *normalized coordinates* are $\tilde{\boldsymbol{x}}_i = \boldsymbol{K}_i^{-1}\bar{\boldsymbol{x}}_i$. The *essential matrix* is $\boldsymbol{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$, where $[\boldsymbol{t}]_\times$ is the skew-symmetric matrix with $[\boldsymbol{t}]_\times \boldsymbol{v} = \boldsymbol{t} \times \boldsymbol{v}$.

Proposition 4.1.2 (Epipolar constraint)**.** Corresponding points satisfy $\tilde{\boldsymbol{x}}_2^\top \boldsymbol{E}\, \tilde{\boldsymbol{x}}_1 = 0$. Hence the match of $\bar{\boldsymbol{x}}_1$ lies on the *epipolar line* $\tilde{\boldsymbol{l}}_2 = \boldsymbol{E}\tilde{\boldsymbol{x}}_1$ in image 2, and the match of $\bar{\boldsymbol{x}}_2$ lies on $\tilde{\boldsymbol{l}}_1 = \boldsymbol{E}^\top \tilde{\boldsymbol{x}}_2$ in image 1. All epipolar lines of an image pass through its *epipole*.

Proof. Up to positive scale, $\tilde{\boldsymbol{x}}_1 \propto \boldsymbol{x}$ and $\tilde{\boldsymbol{x}}_2 \propto \boldsymbol{R}\boldsymbol{x}+\boldsymbol{t}$. Then $(\boldsymbol{R}\boldsymbol{x}+\boldsymbol{t})^\top [\boldsymbol{t}]_\times \boldsymbol{R}\boldsymbol{x} = (\boldsymbol{R}\boldsymbol{x})^\top [\boldsymbol{t}]_\times (\boldsymbol{R}\boldsymbol{x}) + \boldsymbol{t}^\top [\boldsymbol{t}]_\times \boldsymbol{R}\boldsymbol{x} = 0 + 0$, because $\boldsymbol{v}^\top [\boldsymbol{t}]_\times \boldsymbol{v} = 0$ and $\boldsymbol{t}^\top(\boldsymbol{t}\times\boldsymbol{u}) = 0$ for every $\boldsymbol{v}, \boldsymbol{u}$. Scaling does not change a zero. The statement on lines is the observation that $\tilde{\boldsymbol{x}}_2^\top \tilde{\boldsymbol{l}}_2 = 0$ is the equation of a line in image 2. □

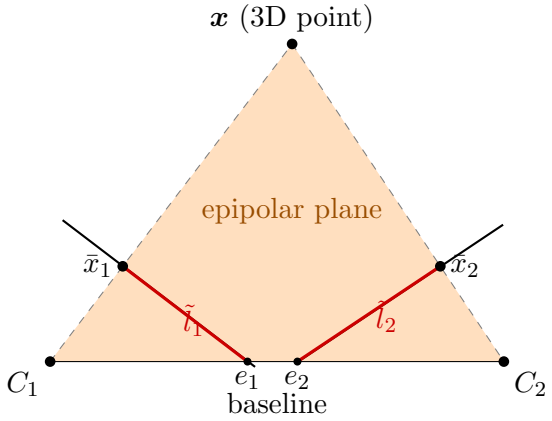


Figure 3: Epipolar geometry (redrawn from slide 7). The camera centres and the 3D point span the epipolar plane; it cuts each image plane in an epipolar line, and the baseline pierces the image planes at the epipoles.

Example 4.1.3 (Why the 1D search matters)**.** For a VGA image ($640 \times 480 \approx 307{,}000$ pixels), an unconstrained correspondence search tests about 3×10^5 candidates per pixel; searching along one epipolar line tests about 640. The saving is $480\times$, i.e. the image height.

Intuition

Given where a pixel sits in image 1, the 3D point can be anywhere along its viewing ray. Seen from camera 2, that ray is a straight line, not a blob: the matching pixel must lie on it. So we ask "which point *on this line* looks the same?" instead of "which point *in the image*?".

4.1.5 Image Rectification *(slides 8)*

If both cameras face exactly the same direction (image planes co-planar), the epipoles are at infinity and the epipolar lines are parallel: the correspondence of a pixel lies on the *same horizontal scanline*. Real rigs are never perfectly aligned, but we can re-warp both images by a *rotation about the camera centres*, mapping them to a common plane parallel to the baseline; this is *rectification*. Because the rotation is about the camera centre, the 3D structure need not be known.

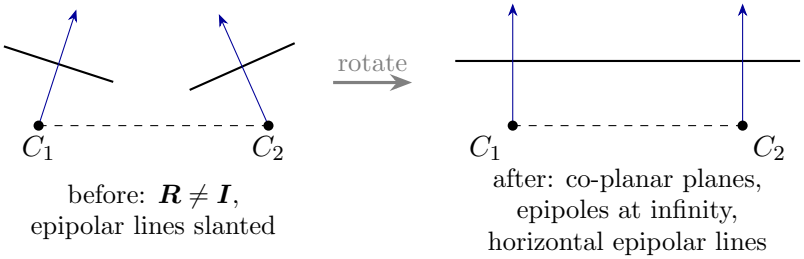


Figure 4: Rectification as a pure rotation of each camera (top view; blue arrows are optical axes). After the rotation both image planes are the same plane, parallel to the baseline.

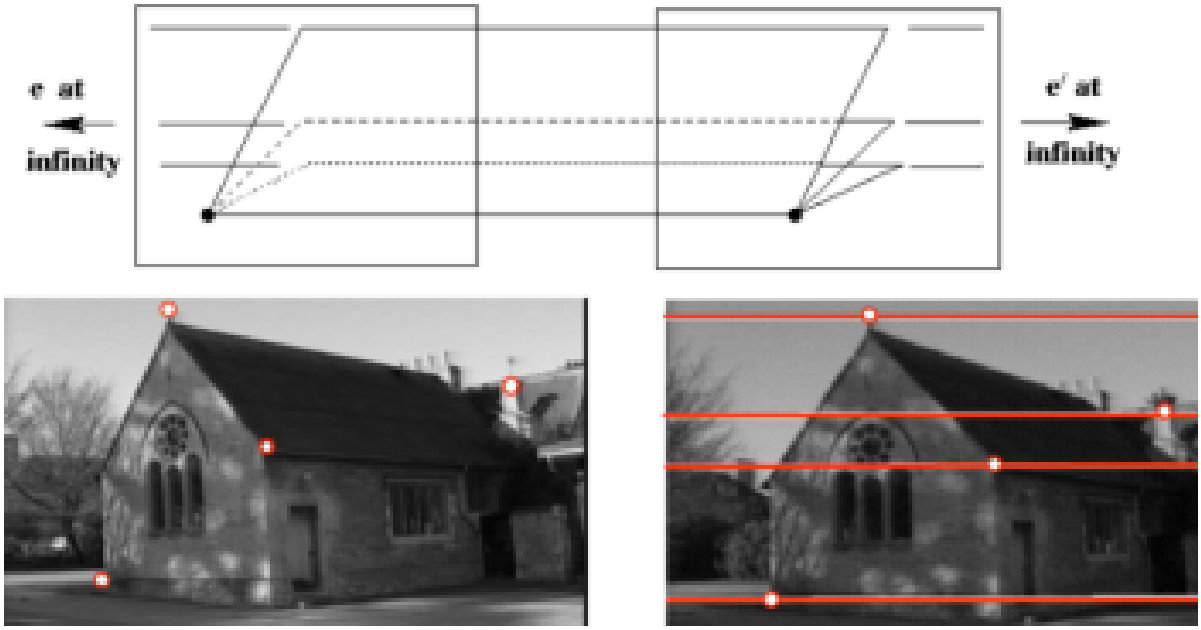


Figure 5: Example: pair of a house before and after rectification.

4.1.6 Rectified Epipolar Constraint *(slides 9)*

Proposition 4.1.4 (Rectified setup)**.** Let $\boldsymbol{K}_1 = \boldsymbol{K}_2 = \boldsymbol{R} = \boldsymbol{I}$ and $\boldsymbol{t} = (t, 0, 0)^\top$. Then

$$\boldsymbol{E} = [\boldsymbol{t}]_\times = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -t \\ 0 & t & 0 \end{pmatrix}, \qquad \bar{\boldsymbol{x}}_2^\top \boldsymbol{E}\, \bar{\boldsymbol{x}}_1 = t\, y_1 - t\, y_2 = 0 \implies y_1 = y_2.$$

Proof. With $\bar{\boldsymbol{x}}_1 = (x_1, y_1, 1)^\top$ we get $\boldsymbol{E}\bar{\boldsymbol{x}}_1 = (0, -t, t\, y_1)^\top$, and $\bar{\boldsymbol{x}}_2^\top (0, -t, t\, y_1)^\top = -t\, y_2 + t\, y_1$. For $t \neq 0$ this vanishes iff $y_1 = y_2$. □

Intuition

The only thing separating the two cameras is a sideways slide. A sideways slide moves every point horizontally and never vertically, so the match is on the same row. This is why stereo code can loop along scanlines.

4.1.7 Calculating the Rectifying Rotation Matrix *(slides 10)*

Lemma 4.1.5 (Rectifying rotation)**.** Let $\boldsymbol{R}, \boldsymbol{t}$ be the relative pose ($\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$). Define

$$\boldsymbol{r}_1 = \frac{-\boldsymbol{R}^\top \boldsymbol{t}}{\|\boldsymbol{R}^\top \boldsymbol{t}\|_2}, \quad \boldsymbol{r}_2 = \frac{[(0,0,1)^\top]_\times \boldsymbol{r}_1}{\|[(0,0,1)^\top]_\times \boldsymbol{r}_1\|_2}, \quad \boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2, \quad \boldsymbol{R}_{\text{rect}} = (\boldsymbol{r}_1, \boldsymbol{r}_2, \boldsymbol{r}_3)^\top.$$

Then $\boldsymbol{R}_{\text{rect}}$ is a rotation and $\boldsymbol{R}_{\text{rect}}\boldsymbol{r}_1 = (1, 0, 0)^\top$: the epipole of camera 1 becomes an ideal point in the x-direction.

Proof. The centre of camera 2 in the frame of camera 1 solves $\boldsymbol{R}\boldsymbol{x} + \boldsymbol{t} = \boldsymbol{0}$, i.e. $\boldsymbol{x} = -\boldsymbol{R}^\top \boldsymbol{t}$; the epipole of image 1 is this direction, so it is $\propto \boldsymbol{r}_1$. By construction $\boldsymbol{r}_2 \perp \boldsymbol{r}_1$ (a cross product is orthogonal to its factors), both are unit vectors, and $\boldsymbol{r}_3 = \boldsymbol{r}_1 \times \boldsymbol{r}_2$ completes a right-handed orthonormal basis. Hence the rows of $\boldsymbol{R}_{\text{rect}}$ are orthonormal and $\boldsymbol{R}_{\text{rect}}\boldsymbol{r}_1 = (\boldsymbol{r}_1^\top \boldsymbol{r}_1, \boldsymbol{r}_2^\top \boldsymbol{r}_1, \boldsymbol{r}_3^\top \boldsymbol{r}_1)^\top = (1, 0, 0)^\top$. □

Algorithm 4.1.6 (Rectification)**.** *Input:* two images, intrinsics $\boldsymbol{K}_1, \boldsymbol{K}_2$ and relative pose $(\boldsymbol{R}, \boldsymbol{t})$.

1. Estimate $\boldsymbol{E}$, decompose it into $\boldsymbol{t}$ and $\boldsymbol{R}$, and build $\boldsymbol{R}_{\text{rect}}$ as in Lemma 4.1.5.
2. Warp image 1: $\tilde{\boldsymbol{x}}'_1 = \boldsymbol{K}\, \boldsymbol{R}_{\text{rect}}\, \boldsymbol{K}_1^{-1}\, \bar{\boldsymbol{x}}_1$.
3. Warp image 2: $\tilde{\boldsymbol{x}}'_2 = \boldsymbol{K}\, \boldsymbol{R}_{\text{rect}}\, \boldsymbol{R}^\top\, \boldsymbol{K}_2^{-1}\, \bar{\boldsymbol{x}}_2$.

($\boldsymbol{K}$ is the common intrinsic matrix of the rectified pair.)

Intuition

Think of $\boldsymbol{r}_1$ as "the direction pointing from camera 1 to camera 2". Rectification spins both cameras so that this direction becomes the image x-axis. Camera 2 first needs undoing its own rotation $\boldsymbol{R}$ (hence $\boldsymbol{R}^\top$) so that both cameras are spun in the same world frame. The recipe for $\boldsymbol{r}_2$ also explains a limit stated later (slide 34): if the baseline points along the optical axis (forward motion), $\boldsymbol{r}_1 \parallel (0,0,1)^\top$, the cross product vanishes and $\boldsymbol{r}_2$ is undefined – planar rectification breaks down.

4.1.8 Rectified Pair and Disparity *(slides 11)*

In a rectified pair the correspondence of a pixel lies in the same image row; the horizontal displacement is the *disparity* $d = x_1 - x_2$ (on the slide 0–160 px, with the match to the left). Disparity is anti-proportional to depth: far points move less than near points.

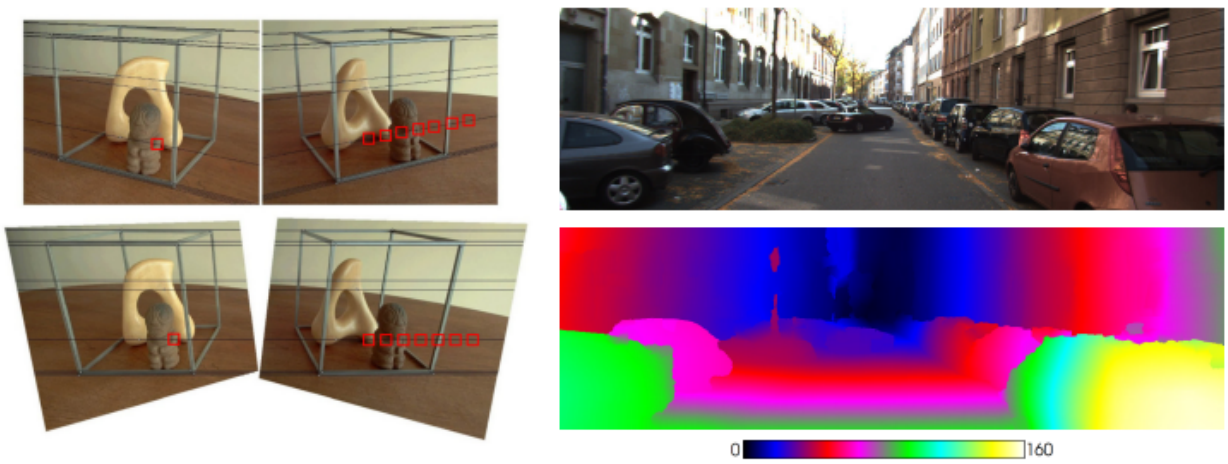


Figure 6: Rectified pair (two lab scenes and a street scene) with colour-coded disparity maps and the 0–160 px colour bar.

4.1.9 Disparity to Depth *(slides 12)*

Theorem 4.1.7 (Triangulation in a rectified pair)**.** Let the cameras have focal length f (in pixels) and baseline b, and let $d = x_1 - x_2$ be the disparity of a point at depth z. Then

$$z = \frac{f\, b}{d}.$$

Proof (slide version). The left and right rays lie in the epipolar plane and intersect at the 3D point; similar triangles give $\frac{z-f}{b-d} = \frac{z}{b}$. Hence $zb - fb = zb - zd$, so $zd = fb$. □

Proof (direct). Place camera 1 at the origin and camera 2 at $(b, 0, 0)$. A point (X, Y, z) projects to $x_1 = fX/z$ and $x_2 = f(X-b)/z$ (principal point at the origin). Subtracting, $d = x_1 - x_2 = fb/z$. □

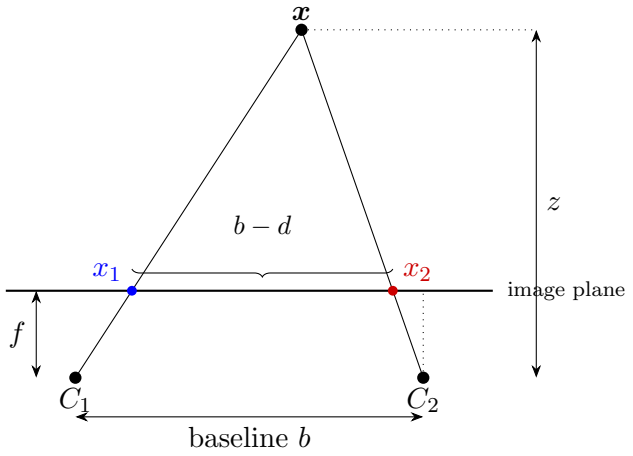


Figure 7: Triangulation (redrawn from slide 12). With $f = 1$, $b = 4$, $z = 4$ in drawing units, each camera measures its own image coordinate ($x_1 = 0.65$ from C_1, $x_2 = -0.35$ from C_2), so $d = x_1 - x_2 = 1 = fb/z$; the similar triangles $(f,\ b-d)$ and $(z,\ b)$ give the formula.

Intuition

Close one eye and point at your thumb, then switch eyes: the thumb jumps a lot. A distant tree barely moves. Disparity *is* that jump, and $z = fb/d$ converts "how much it jumped" into "how far it is". Doubling the baseline doubles every jump, so a given pixel precision resolves finer depth steps.

4.1.10 Worked Example: Baseline and Depth Error *(slides 13)*

Theorem 4.1.8 (Depth error)**.** A disparity error δd causes, to first order, a depth error

$$\delta z \approx \left|\frac{\partial z}{\partial d}\right| \delta d = \frac{fb}{d^2}\,\delta d = \frac{z^2}{fb}\,\delta d.$$

Proof. Differentiate $z = fb\,d^{-1}$: $\partial z/\partial d = -fb/d^2$. Substitute $d = fb/z$ to get $fb/d^2 = z^2/(fb)$. □

Corollary 4.1.9. The relative error $\delta z/z = z\,\delta d/(fb)$ grows linearly with z; doubling b halves δz at every depth; if the smallest measurable disparity is $d_{\min}$, the usable range is $z_{\max} = fb/d_{\min}$.

Example 4.1.10 (Car-like rig, $f \approx 721$ px, $b \approx 0.54$ m, $fb \approx 389$ px·m, $\delta d = 1$ px)**.**

depth z	disparity d	depth error δz
10 m	38.9 px	0.26 m
20 m	19.5 px	1.03 m
40 m	9.7 px	4.11 m

(Beyond the slide: the linearisation is first order; with $d - 1$ at $z = 40$ m the exact value is $389/8.73 - 40 \approx 4.6$ m.)

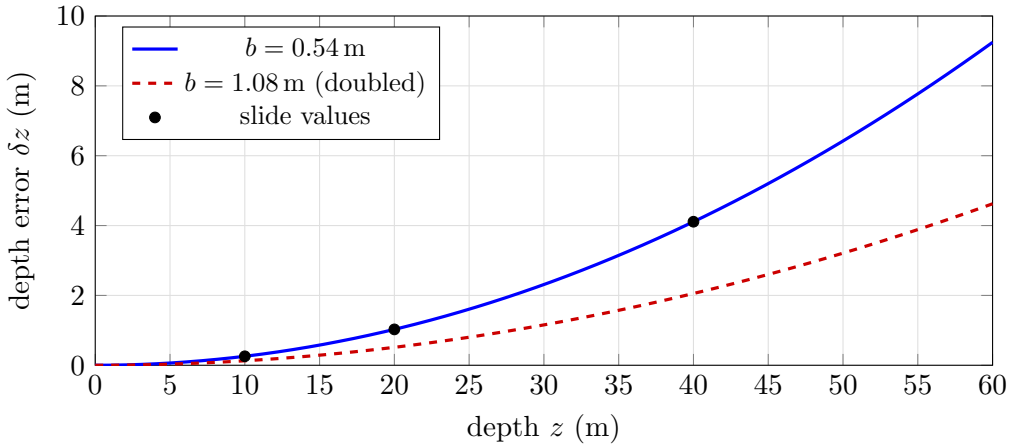


Figure 8: Depth error for a one-pixel disparity error, $\delta z = z^2 \delta d/(fb)$ with $f = 721\,\mathrm{px}$. The error is quadratic in depth; doubling the baseline halves it everywhere (new figure, not on the slides).

Intuition

At 40 m the disparity is only 9.7 px. Being off by one pixel means being off by about 10 % of the measurement, and that 10 % turns into 4 m. Near the camera the same one-pixel slip costs a few centimetres. A wider baseline helps the arithmetic, but the two views then differ more and matching gets harder – the central design tension of stereo.

4.2 Block Matching *(slides 14–21)*

4.2.1 Correspondence Ambiguity *(slides 15)*

How do we decide that two image points correspond? Assume they look "similar". But a single pixel does not reveal local structure (ambiguity), so we compare a small patch – and even then the task is difficult (repetitive or textureless regions, changing appearance).



Figure 9: Three grayscale views of an animal with red boxes marking a tail/patch whose appearance changes

4.2.2 Similarity Metrics *(slides 16)*

Take two $K \times K$ windows flattened to vectors $\boldsymbol{w}_L, \boldsymbol{w}_R \in \mathbb{R}^{K^2}$.

Definition 4.2.1 (SSD and ZNCC)**.**

$$\mathrm{SSD}(x,y,d) = \|\boldsymbol{w}_L(x,y) - \boldsymbol{w}_R(x-d,y)\|_2^2,$$
$$\mathrm{ZNCC}(x,y,d) = \frac{(\boldsymbol{w}_L - \mu_L)^\top (\boldsymbol{w}_R - \mu_R)}{\|\boldsymbol{w}_L - \mu_L\|_2 \, \|\boldsymbol{w}_R - \mu_R\|_2},$$

with μ_L, μ_R the means of the two windows (arguments omitted on the right). SSD is a *cost* (minimise); ZNCC is a *similarity* in $[-1, 1]$ (maximise).

Lemma 4.2.2 (Gain/bias invariance)**.** For $a > 0$ and $b \in \mathbb{R}$, $\mathrm{ZNCC}(a\boldsymbol{w}_L + b\mathbf{1}, \boldsymbol{w}_R) = \mathrm{ZNCC}(\boldsymbol{w}_L, \boldsymbol{w}_R)$. Moreover, for the normalised windows $\hat{\boldsymbol{w}} = (\boldsymbol{w} - \mu)/\|\boldsymbol{w} - \mu\|_2$ one has $\mathrm{SSD}(\hat{\boldsymbol{w}}_L, \hat{\boldsymbol{w}}_R) = 2 - 2\ \mathrm{ZNCC}(\boldsymbol{w}_L, \boldsymbol{w}_R)$.

Proof. The mean of $a\boldsymbol{w}_L + b\mathbf{1}$ is $a\mu_L + b$, so the centred vector is $a(\boldsymbol{w}_L - \mu_L)$ and its norm $a\|\boldsymbol{w}_L - \mu_L\|$ (as $a > 0$); a cancels in the ratio. For unit vectors $\boldsymbol{u}, \boldsymbol{v}$: $\|\boldsymbol{u} - \boldsymbol{v}\|^2 = 2 - 2\boldsymbol{u}^\top \boldsymbol{v}$, and $\hat{\boldsymbol{w}}_L^\top \hat{\boldsymbol{w}}_R = \mathrm{ZNCC}$. □

Example 4.2.3. Let $\boldsymbol{w}_L = (10, 20, 30, 40)$ and $\boldsymbol{w}_R = \boldsymbol{w}_L + 100 = (110, 120, 130, 140)$ (same texture, brighter exposure). Then $\mathrm{SSD} = 4 \cdot 100^2 = 40{,}000$ – a terrible score – but $\mathrm{ZNCC} = 1$, a perfect match. For the reversed window $(40, 30, 20, 10)$, $\mathrm{ZNCC} = -1$.

Intuition

SSD asks "are the numbers equal?", ZNCC asks "do the two patches have the same *pattern of ups and downs*?". When the two cameras have different gain or exposure, only the second question is the right one. Other metrics exist (Szeliski, Ch. 12.3; Hirschmüller and Scharstein, TPAMI 2009).

4.2.3 Block Matching *(slides 17)*

Algorithm 4.2.4 (Block matching with winner-takes-all)**.** *Input:* a rectified image pair.

1. Choose a disparity range $[0, d_{\max}]$.
2. For every pixel (x, y) and every d, compute the matching cost $C(x, y, d)$ (e.g. SSD), giving a *cost volume*.
3. Winner-takes-all: $\hat{d}(x, y) = \arg\min_{d \in [0, d_{\max}]} C(x, y, d)$.
4. Repeat with the roles of the images exchanged and apply the left-right consistency check to remove outliers.

Naively this costs $O(WHD\,K^2)$ per image.

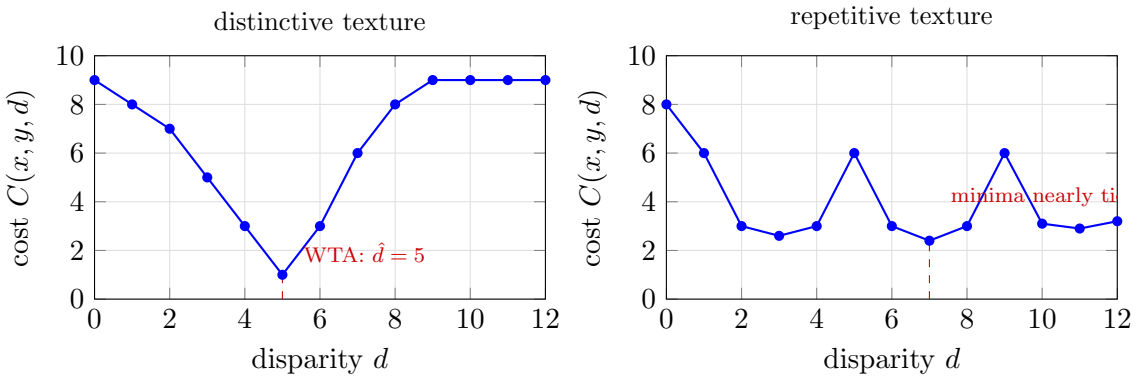


Figure 10: Illustrative cost curves along one scanline (values invented for illustration, not measured). Left: a clear unique minimum. Right: repeated texture produces several near-equal minima, so noise can make winner-takes-all pick the wrong one.

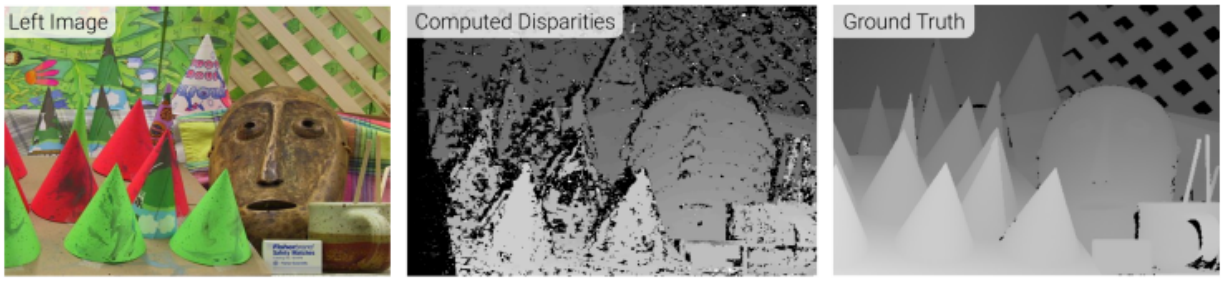


Figure 11: Left image, computed disparities (noisy, with holes) and ground truth

Intuition

Block matching is a "find-my-twin" game on one row: slide a small window of the left image along the row of the right image and keep the position with the lowest cost. Each pixel decides alone – nobody checks whether neighbours agree – which is why the output is noisy and why later sections add regularisation and learning.

4.2.4 Block Matching: Half Occlusions *(slides 18)*

A *half-occlusion* is a region visible in the left image but not in the right one (red region on the slide).

Proposition 4.2.5 (Width of the half-occluded strip)**.** Let a fronto-parallel foreground with disparity d_f stand in front of a background with disparity $d_b < d_f$. The strip of background that is visible in the left image but hidden in the right image lies immediately left of the foreground's left edge in the left image and has width $d_f - d_b$ pixels.

Proof. Let the foreground occupy $[a, e]$ in the left image; it appears at $[a - d_f, e - d_f]$ in the right image. A background pixel x of the left image appears at $x - d_b$ in the right image. It is hidden in the right image iff its right-image position $x - d_b$ falls inside the foreground's footprint $[a - d_f, e - d_f]$ while $x < a$ (it is not itself foreground). The condition $x - d_b \geq a - d_f$ reads $x \geq a - (d_f - d_b)$, so the hidden background pixels are exactly $x \in [a - (d_f - d_b),\ a)$, an interval of length $d_f - d_b$. □

Example 4.2.6. With the rig of the previous section, a car at 10 m ($d_f = 38.9$ px) in front of a wall at 40 m ($d_b = 9.7$ px) hides a strip 29.2 px wide next to it. No matching cost can find a twin there, because the twin does not exist.

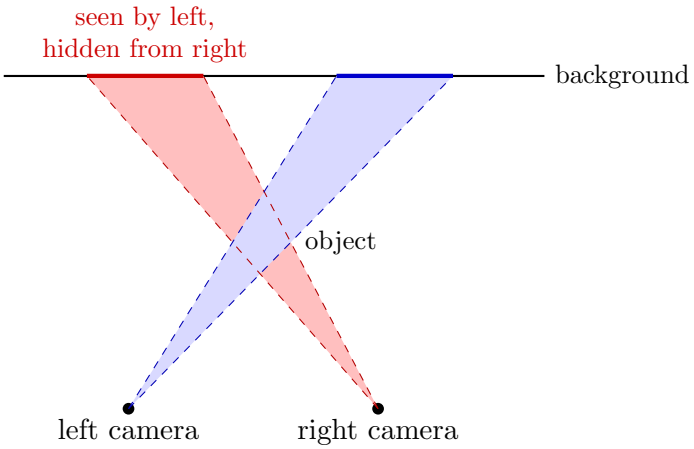


Figure 12: Half-occlusion (top view, redrawn from slide 18). Red: background strip hidden from the right camera but visible to the left one; blue: strip hidden from the left camera.

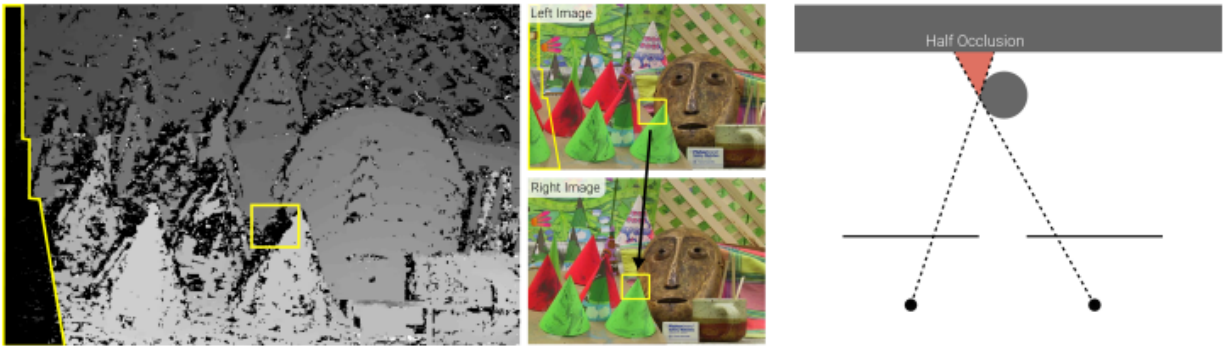


Figure 13: Left image with the half-occluded region marked in red, zoomed left/right crops, and the occlusion sketch

4.2.5 Block Matching: Assumption Violations *(slides 19)*

Block matching assumes that *all pixels inside the window are displaced by the same d*: the *fronto-parallel assumption*. It is often wrong: slanted surfaces deform perspectively between views, and at disparity discontinuities the window content changes differently on the two sides.

Proposition 4.2.7 (Disparity on a plane is affine)**.** For a plane $\boldsymbol{n}^\top \boldsymbol{X} = h$ ($h \neq 0$) in the rectified camera frame, $1/z$ is an affine function of the pixel (x, y); hence so is the disparity $d = fb/z$.

Proof. A pixel $\bar{\boldsymbol{x}}$ sees $\boldsymbol{X} = z\,\boldsymbol{K}^{-1}\bar{\boldsymbol{x}}$. Substituting in the plane equation, $z\,\boldsymbol{n}^\top \boldsymbol{K}^{-1}\bar{\boldsymbol{x}} = h$, so $1/z = \boldsymbol{n}^\top \boldsymbol{K}^{-1}\bar{\boldsymbol{x}}/h$, which is linear in $\bar{\boldsymbol{x}} = (x, y, 1)^\top$. Multiply by fb. □

Intuition

A fronto-parallel wall has constant disparity; a floor or a slanted wall has a disparity that *ramps* across the window. If the ramp is 0.1 px per pixel and the window is 11 pixels wide, the left and right windows disagree by about 1.1 px at the edges, enough to hurt the match. Larger windows suffer more.

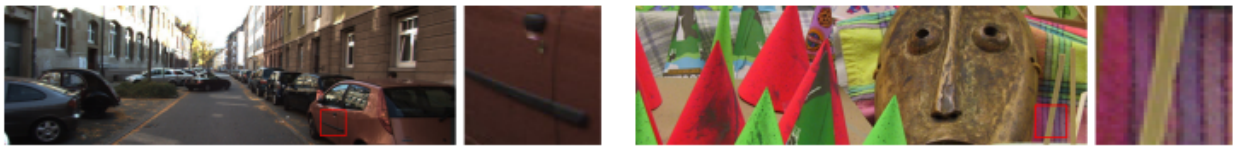


Figure 14: Street scene and Teddy-like scene crops showing perspective deformation of windows on slanted surfaces

4.2.6 Effect of Window Size *(slides 20)*

Trade-off. Small windows give matching ambiguities and noisy disparity maps. Larger windows give smoother results but lose detail and suffer *border bleeding*: when a window straddles a strong edge, left and right patches get the same disparity because of the dominant edge; the weakly textured background (red on the slide) lacks gradients to overcome it.

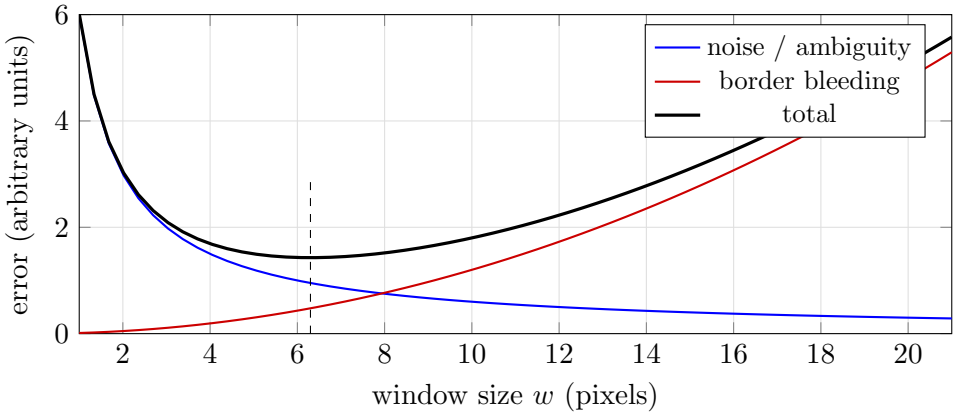


Figure 15: Conceptual bias–variance-style trade-off in window size (illustrative curves, not measured; the slides show it with 5×5 vs 11×11 results).



Figure 16: Disparity results with 5×5 and 11×11 windows, with "border bleeding" regions boxed

4.2.7 Left-Right Consistency Test *(slides 21)*

Definition 4.2.8 (Left-right consistency)**.** Let $d_L(x, y)$ be the disparity from matching with the left image as reference, and $d_R(x', y)$ the one with the right image as reference. A pixel is *consistent* (kept) if

$$\big|\, d_L(x, y) - d_R\big(x - d_L(x, y),\, y\big)\big| \le \tau,$$

typically with a small threshold τ (about one pixel); otherwise it is marked invalid. In words: follow the match from left to right and back; you must land where you started (a cycle).

Example 4.2.9. Left disparities at $x = 2, \ldots, 6$ are $d_L = (2, 2, 2, 4, 2)$, so these pixels map to right pixels $x' = x - d_L = (0, 1, 2, 1, 4)$. Suppose $d_R(0) = d_R(1) = d_R(2) = d_R(4) = 2$. Then the differences $|d_L - d_R(x')|$ are $(0, 0, 0, 2, 0)$: with $\tau = 1$ the pixel $x = 5$ is rejected. Note that $x = 3$ and $x = 5$ both claim right pixel 1, which is impossible for a surface that is visible in both views.

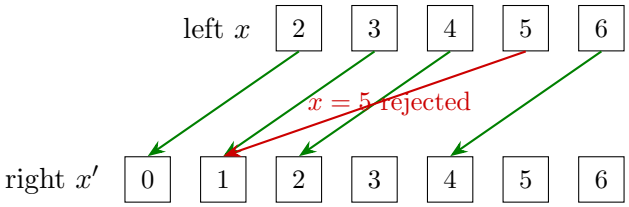


Figure 17: Toy left-to-right matches from the example (new figure). Two left pixels landing on the same right pixel flags an inconsistency.

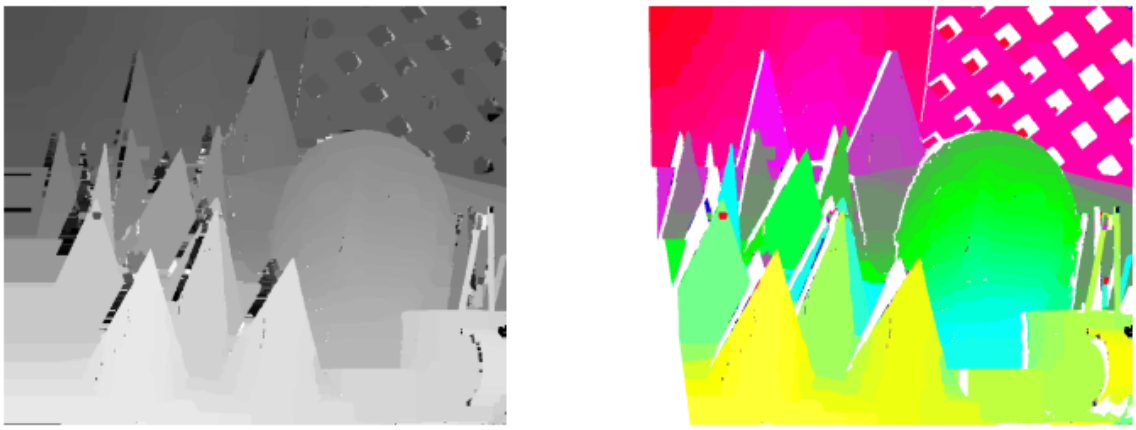


Figure 18: Left and right disparity maps used for the consistency check

Intuition

Two witnesses must tell the same story: if "left says my twin is 4 px away" but "that twin says its twin is 2 px away", at least one of them is lying. Half-occluded pixels fail this test automatically, since the supposed twin belongs to something else.

4.3 Siamese Networks *(slides 22–28)*

4.3.1 Siamese Networks for Stereo Matching *(slides 23)*

Hand-crafted features and similarity metrics do not capture all relevant geometric and radiometric invariances or occlusion patterns; the world is too complex to specify them by hand. Zbontar and LeCun (JMLR 2016) therefore treat *matching cost computation as an image classification problem*:

- train a CNN patch-wise on image pairs with ground-truth disparity (e.g. KITTI Stereo 2015);

- compute features for each of the two images with the learned model;
- correlate features between the two images (dot product);
- take the maximum per pixel (winner-takes-all) or run a global optimisation.

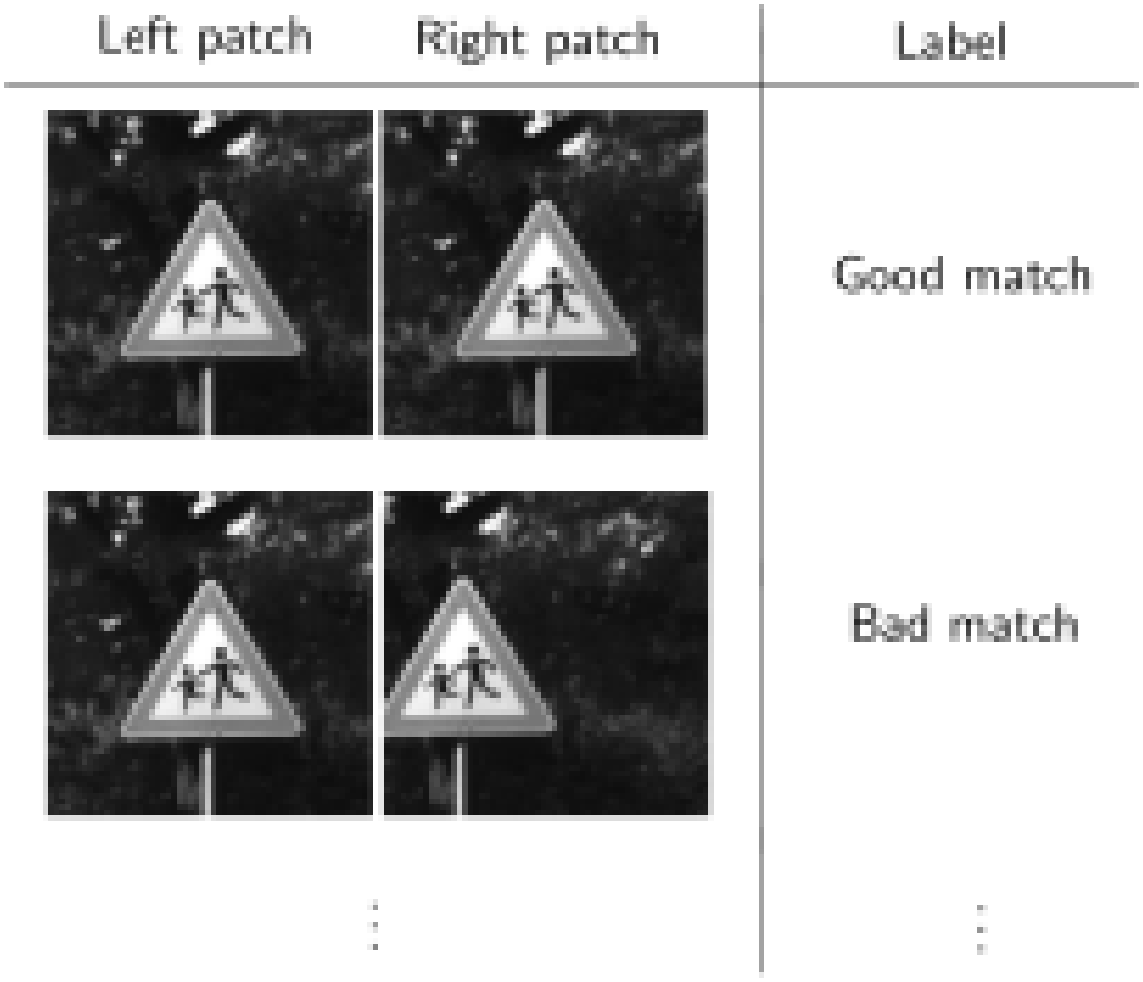


Figure 19: Small table of left patch / right patch / label (good match, bad match) with traffic-sign crops

4.3.2 Siamese Network Architecture *(slides 24)*

Two variants are compared on the slide.

Learned similarity	**Cosine similarity**
learn features *and* the similarity metric; potentially more expressive; slow ($W \times H \times D$ MLP evaluations)	learn features and apply a dot product; the features must do the heavy lifting; fast matching (no network evaluation)

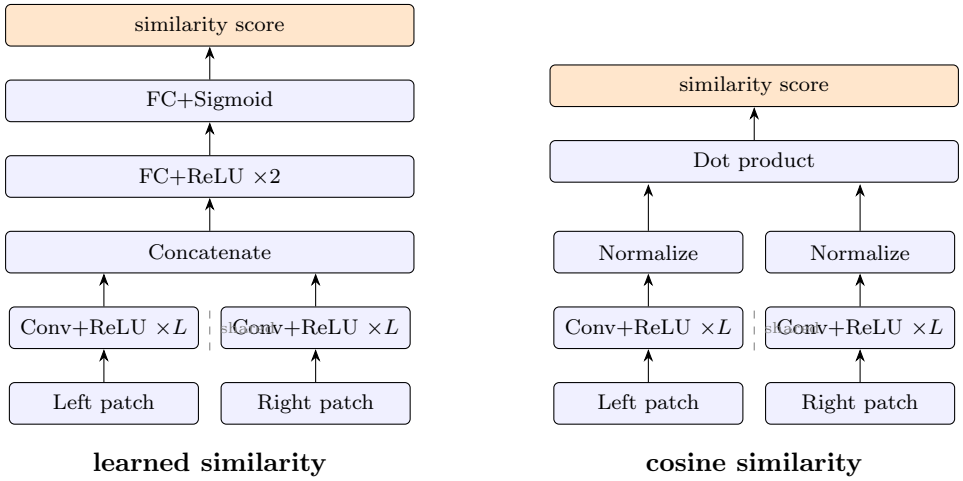


Figure 20: The two Siamese stereo architectures of Zbontar and LeCun (redrawn from slide 24; layer counts are schematic). Both towers share weights.

Example 4.3.1 (Why the cosine head is fast)**.** For a KITTI-sized image (1242×375) and $D = 228$ candidate disparities, the learned-similarity head needs $WHD \approx 1.06 \times 10^8$ network evaluations. The cosine head runs each tower once per image, then each cost is just a dot product of two stored feature vectors.

Intuition

Learned similarity is like hiring an expert who inspects every candidate pair. Cosine similarity is like giving every patch a fingerprint once and then comparing fingerprints: much cheaper, but the fingerprint must be good enough that a plain dot product suffices.

4.3.3 Training Triplets *(slides 25)*

Definition 4.3.2 (Training triplet)**.** A triplet is $\big(\boldsymbol{w}_L(\boldsymbol{x}_L^{\text{ref}}),\ \boldsymbol{w}_R(\boldsymbol{x}_R^{\text{neg}}),\ \boldsymbol{w}_R(\boldsymbol{x}_R^{\text{pos}})\big)$ with

$$\boldsymbol{x}_R^{\text{neg}} = (x_L^{\text{ref}} - d + o_{\text{neg}},\, y_L^{\text{ref}}), \qquad \boldsymbol{x}_R^{\text{pos}} = (x_L^{\text{ref}} - d + o_{\text{pos}},\, y_L^{\text{ref}}),$$

where d is the ground-truth disparity, $o_{\text{neg}} \sim \mathcal{U}(\{-N_{\text{hi}}, \dots, -N_{\text{lo}}, N_{\text{lo}}, \dots, N_{\text{hi}}\})$ and $o_{\text{pos}} \sim \mathcal{U}(\{-P_{\text{hi}}, \dots, P_{\text{hi}}\})$. Typical values: $P_{\text{hi}} = 1$, $N_{\text{lo}} = 3$, $N_{\text{hi}} = 6$.

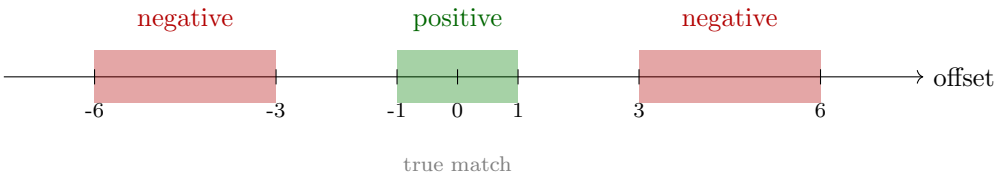


Figure 21: Where positive and negative right patches are sampled relative to the true match (new figure; ranges $P_{\text{hi}} = 1$, $N_{\text{lo}} = 3$, $N_{\text{hi}} = 6$ from the slide).

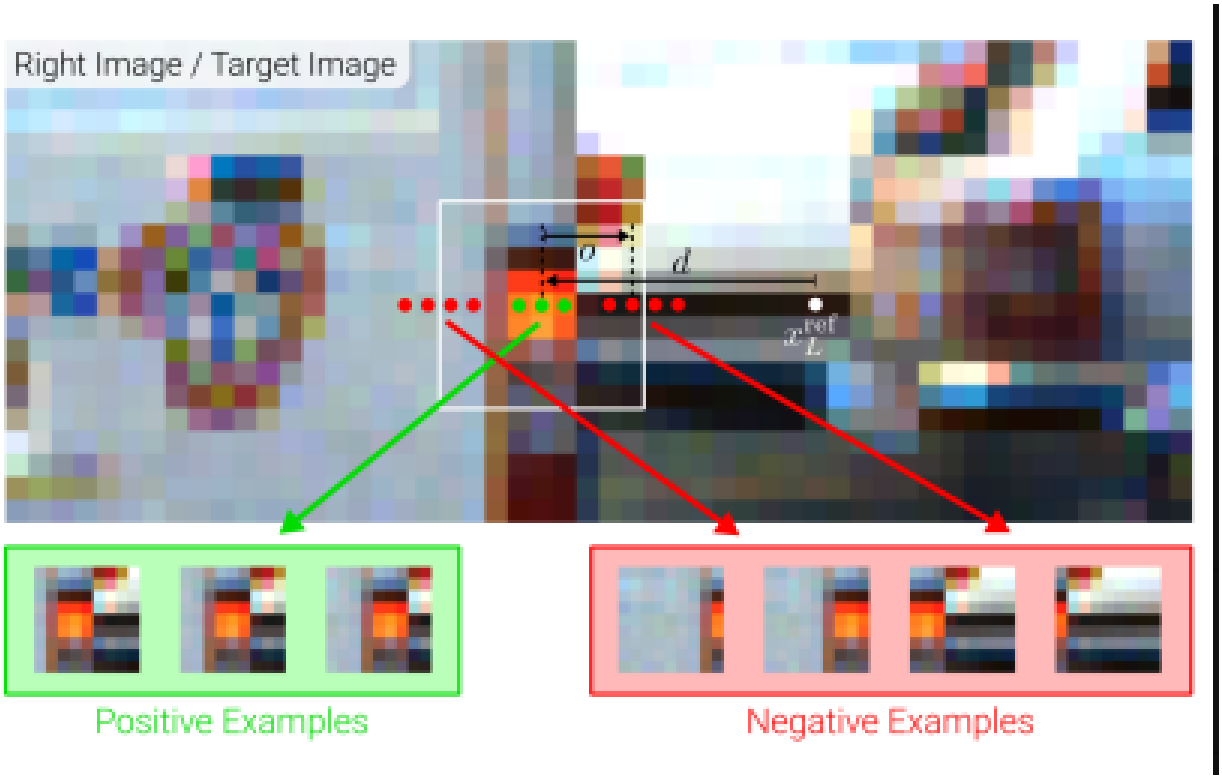


Figure 22: Zoomed right image with a window and rows of green "positive" crops and red "negative" crops

Intuition

Negatives are not random patches: they come from just a few pixels away from the truth (3 to 6 px), so they look similar and the network must learn the fine cues. Positives are allowed ± 1 px of slack, because a match is never pixel-perfect. Negatives closer than N_{lo} are excluded since they would be nearly indistinguishable from the positives. This is a mild form of *hard negative mining*. With these values there are 3 positive and 8 negative offsets to draw from.

4.3.4 Hinge Loss *(slides 26)*

Definition 4.3.3 (Hinge (margin) loss)**.** With s_+ and s_- the network scores of the positive and negative example and margin $m > 0$,

$$\mathcal{L} = \max(0,\, m + s_- - s_+).$$

Lemma 4.3.4. $\mathcal{L} = 0$ iff $s_+ \geq s_- + m$. When $\mathcal{L} > 0$, $\partial\mathcal{L}/\partial s_+ = -1$ and $\partial\mathcal{L}/\partial s_- = +1$: gradient descent raises s_+ and lowers s_- at the same rate.

Example 4.3.5 ($m = 0.2$, the value used in the paper)**.**

s_+	s_-	$m + s_- - s_+$	loss	meaning
0.9	0.3	-0.4	0	clearly separated, nothing to learn
0.6	0.5	0.1	0.1	right order but too close: push apart
0.4	0.7	0.5	0.5	wrong order: large correction

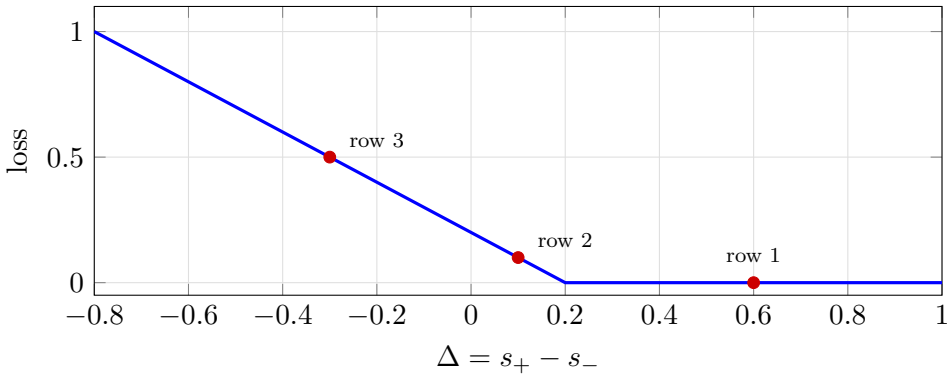


Figure 23: Hinge loss with $m = 0.2$ as a function of the score gap; red dots are the three rows of the table (new figure).

Intuition

The loss does not demand that the positive scores high in absolute terms, only that it beats the negative by a safety margin. Once it does, the example is "solved" and stops contributing – training focuses its effort on the pairs that are still confusable.

4.3.5 Comparisons *(slides 27)*

The slide compares, on a KITTI street scene, the left input, standard block matching, the Siamese-network matching cost, a semiglobal-matching (SGM, "optimize") variant, the left and right disparity maps, and the left-right consistency test. The qualitative message: learned costs are far less noisy than SAD/SSD-style block matching, and adding a global optimisation (SGM) on top improves them further.

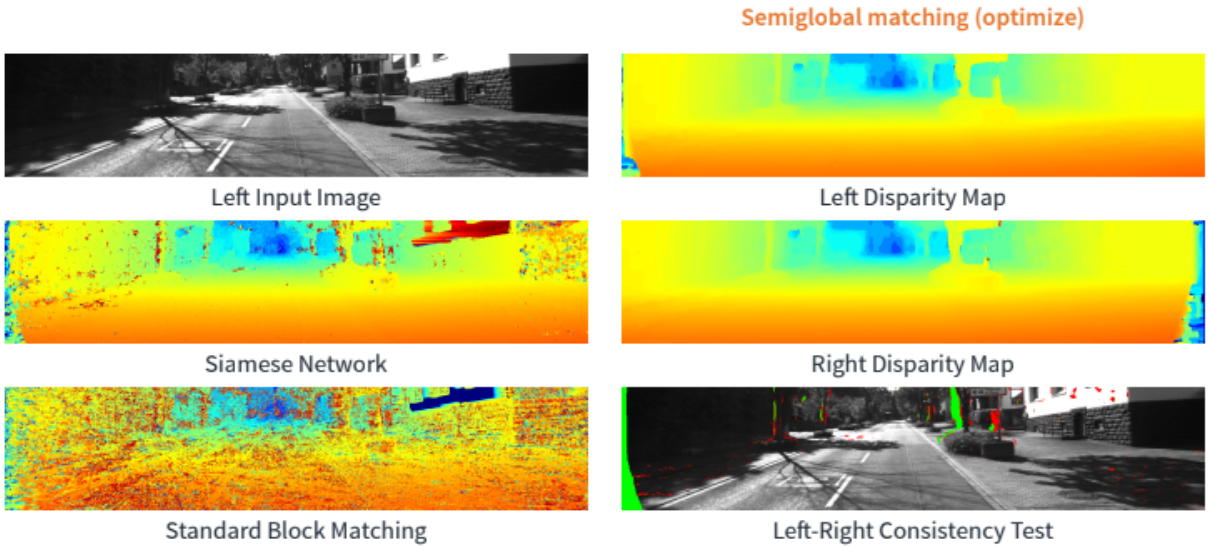


Figure 24: Grid of six images: left input, Siamese network, standard block matching, left disparity (SGM), right disparity, left-right consistency test

4.3.6 Similarity Constraint: Failure Cases *(slides 28)*

The underlying assumption is that *corresponding regions look similar and non-corresponding regions look different*. It fails for: **repetitions** (several patches look alike), **textureless surfaces** (everything looks alike), **occlusions** (the twin does not exist) and **non-Lambertian surfaces** (the same point looks different from two views, e.g. reflections). The fix suggested on the slide is spatial regularisation (smoothness priors, cf. Huang, Lee and Mumford on range-image statistics).

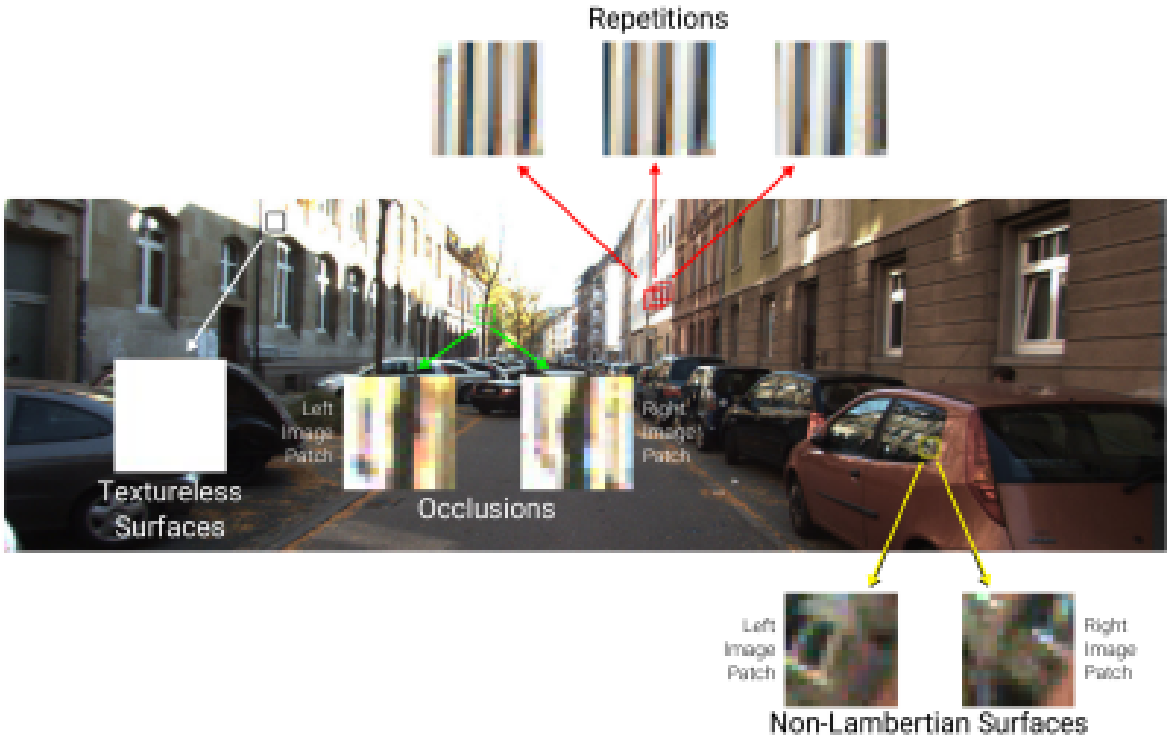


Figure 25: Street photo annotated with "Repetitions", "Textureless surfaces", "Occlusions" and "Non-Lambertian surfaces" crops

Intuition

A matching cost only looks at one pixel (or patch) at a time. When the local evidence is ambiguous or absent, it needs a prior about how the world behaves – "neighbouring pixels usually have similar depth" – to choose between candidates.

4.4 End-to-End Learning *(slides 29–31)*

4.4.1 DispNet and Synthetic Data *(slides 30)*

- DispNet (Mayer et al., CVPR 2016) was the first end-to-end trained deep network for stereo.
- It uses a U-Net-like architecture with skip connections to retain details.
- A *correlation layer* compares left and right feature maps (the slide notes: 40 px displacement at feature resolution corresponds to 160 px in the input image, i.e. a stride of 4).
- Training uses a multi-scale loss (disparity error in pixels) and *curriculum learning* (easy to hard).
- Pre-train on large *synthetic* datasets (cheap annotations: FlyingThings3D, Monkaa), then fine-tune on a little real data (expensive annotations).

Definition 4.4.1 (1D correlation layer *(general form, not written on the slide)*)**.** For feature maps f_L, f_R and maximum displacement D, a correlation layer outputs the volume $c(x, y, d) = \langle f_L(x, y), f_R(x - d, y) \rangle$ for $d = 0, \ldots, D$.

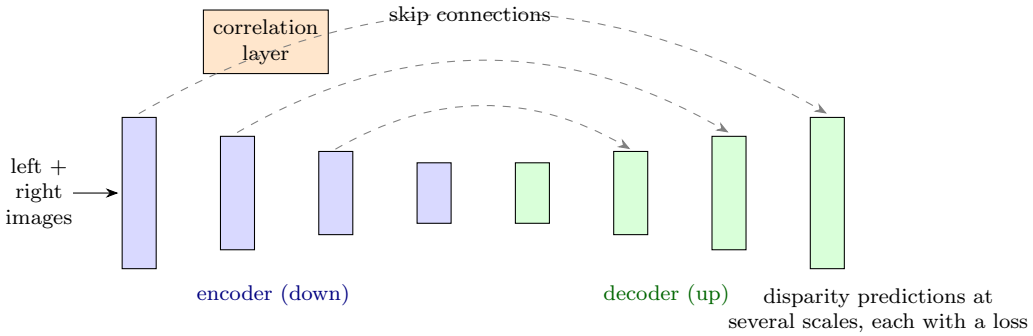


Figure 26: Schematic of a DispNet-style encoder–decoder with skip connections (new schematic, not a reproduction of the slide's network figure).

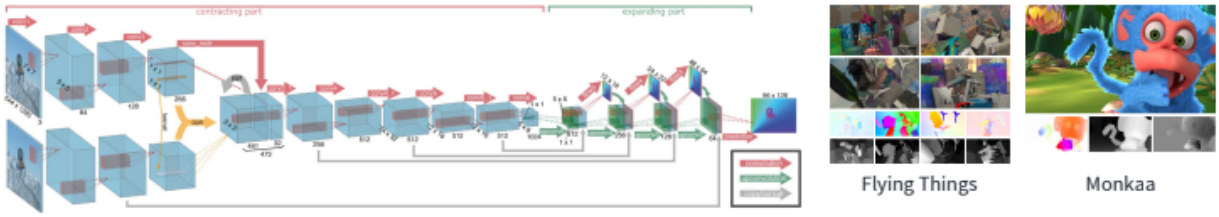


Figure 27: DispNet network diagram plus FlyingThings3D and Monkaa sample images

Intuition

Hand-designed block matching has no way to "learn" that a textureless wall continues smoothly. A network trained on millions of synthetic examples sees every ambiguity, and the U-Net structure lets coarse levels settle the global layout while fine levels sharpen the edges. The synthetic→real recipe exists because perfect ground truth is trivial for a renderer and very costly for real scenes.

4.4.2 GC-Net *(slides 31)*

Key idea (Kendall et al., ICCV 2017): build a *disparity cost volume* and apply *3D convolutions* to it. The learned matching cost $c_\theta(d)$ is converted to a disparity with a differentiable expectation.

Definition 4.4.2 (Soft argmin)**.**

$$\hat{d} = \mathbb{E}[d] = \sum_{d=0}^{d_{\max}} p(d)\, d, \qquad p(d) = \operatorname{softmax}_d\big(-c_\theta(d)\big) = \frac{e^{-c_\theta(d)}}{\sum_k e^{-c_\theta(k)}}.$$

Proposition 4.4.3. (i) $\hat{d} \in [0, d_{\max}]$ and $\hat{d}$ may be non-integer (sub-pixel). (ii) $\hat{d}$ is differentiable in the costs with $\dfrac{\partial \hat{d}}{\partial c(k)} = -p(k)\,\big(k - \hat{d}\big)$.

Proof. (i) p is a probability vector, so $\hat{d}$ is a convex combination of the integers $0, \dots, d_{\max}$. (ii) $\partial p(d)/\partial c(k) = -p(d)(\delta_{dk} - p(k))$, hence $\partial \hat{d}/\partial c(k) = \sum_d d\,[-p(d)(\delta_{dk} - p(k))] = -p(k)k + p(k)\hat{d}$. □

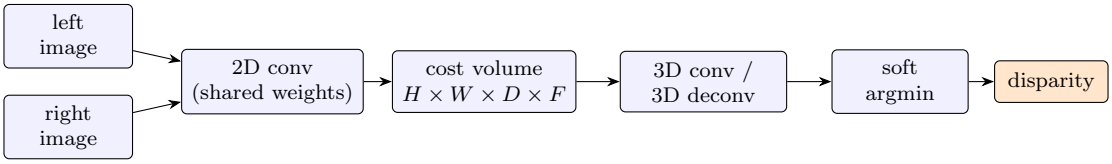


Figure 28: GC-Net pipeline (redrawn from slide 31).

Example 4.4.4 (Soft argmin on two cost profiles, $d = 0, \ldots, 4$)**.**

	costs $c(d)$	$p(d)$	$\hat{d}$
unimodal	$(3, 1, 0, 1, 3)$	$(0.027, 0.200, 0.545, 0.200, 0.027)$	2.0
bimodal	$(0, 4, 4, 4, 0)$	$(0.487, 0.009, 0.009, 0.009, 0.487)$	2.0

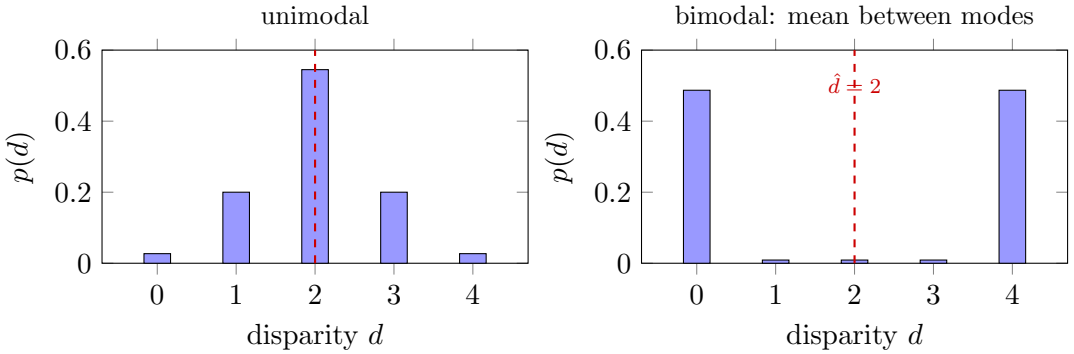


Figure 29: Soft-argmin outputs (red dashed line) for the two profiles of the example. In the bimodal case the expectation is a disparity that neither mode supports (new figure).

Example 4.4.5 (Memory of a 3D volume (illustrative sizes))**.** A volume with $H = 256$, $W = 512$, $D = 48$ (disparities at quarter resolution) and $F = 32$ float32 channels takes $256 \cdot 512 \cdot 48 \cdot 32 \cdot 4 \approx 0.8\,\text{GB}$, before any activations of the 3D convolutions are stored for back-propagation.

Intuition

An ordinary argmin is a hard decision with no gradient. The soft argmin "votes" with weights given by the softmax, which makes the whole network trainable end to end and allows sub-pixel answers. Its weakness is visible in the bimodal row: at an object boundary the cost has two valleys (foreground and background); the expectation averages them and invents a surface in between – the *smoothness bias* that SMD-Nets later address (backup slides). 3D convolutions add context along the disparity axis at the price of memory ("slightly better performance but large memory requirements").

4.5 Stereo Without Calibration *(slides 32–45)*

4.5.1 The Cost of Calibration *(slides 33)*

Calibration is measured *offline*, with a target, before any image pair is matched. It drifts with temperature, vibration and refocusing; a small rotation error leaves corresponding points on different rows after rectification. Two arbitrary photographs come with no calibration at all and cannot be rectified. The learned stereo networks of Sec. 4.4 inherit all of this: rectified input and a fixed disparity range. (Calibration is the topic of the next lecture.)

4.5.2 Limits of Rectification and Local Matching *(slides 34)*

- Forward motion puts the epipole *inside* the image, where planar rectification breaks down (compare the remark after Lemma 4.1.5).
- Large rotations between the views distort the rectified images heavily.
- A wide baseline lowers the depth error (Theorem 4.1.8) but changes the appearance of every patch, so local matching fails.

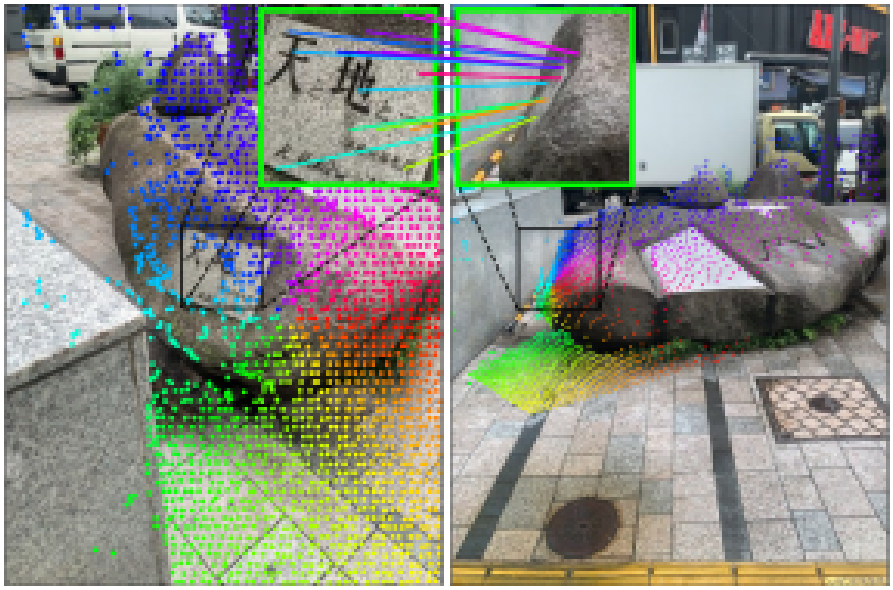


Figure 30: Dense correspondences where camera motion degrades visual similarity

4.5.3 Pointmaps *(slides 35)*

Definition 4.5.1 (Pointmap)**.** A pointmap $\boldsymbol{X}^{a,1} \in \mathbb{R}^{H\times W\times 3}$ assigns to each pixel $\boldsymbol{x}_s$ of image I_a, $a \in \{1,2\}$, its 3D point $\boldsymbol{X}^{a,1}[\boldsymbol{x}_s] \in \mathbb{R}^3$ expressed in the frame of camera 1 (the second index names the frame). For image 1 with depth z, intrinsics $\boldsymbol{K}_1$ and augmented pixel $\bar{\boldsymbol{x}}_s$:

$$\boldsymbol{X}^{1,1}[\boldsymbol{x}_s] = z\,\boldsymbol{K}_1^{-1}\bar{\boldsymbol{x}}_s.$$

DUSt3R regresses $\hat{\boldsymbol{X}}^{1,1}$ and $\hat{\boldsymbol{X}}^{2,1}$ from two images – both in the frame of camera 1 (the hat marks a prediction). Two pixels correspond when their 3D points coincide: *matching becomes a 3D problem.*

Proposition 4.5.2 (Disparity map as a special case)**.** For a rectified pair with focal length f, principal point (c_x, c_y) and baseline b, a pixel (x, y) with disparity d has pointmap entry $\boldsymbol{X}^{1,1} = \dfrac{b}{d}\,(x - c_x,\ y - c_y,\ f)^\top$.

Proof. $z = fb/d$ by Theorem 4.1.7, and $\boldsymbol{K}_1^{-1}\bar{\boldsymbol{x}} = ((x-c_x)/f, (y-c_y)/f, 1)^\top$; multiply. □

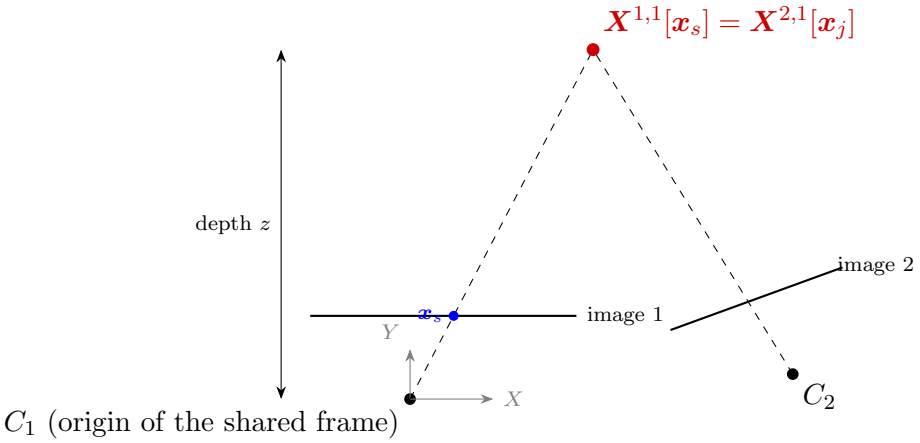


Figure 31: Pointmaps (new figure). Pixel $\boldsymbol{x}_s$ in image 1 and pixel $\boldsymbol{x}_j$ in image 2 are *the same 3D point* expressed in camera 1's frame; this is the correspondence criterion.

Intuition

A disparity map stores one number per pixel and only makes sense along a rectified row. A pointmap stores the whole 3D position in a frame shared by both images, so the notion "these two pixels see the same thing" becomes simply "their 3D points are equal" – no rows, no calibration of the pair needed to state it.

4.5.4 The DUSt3R Architecture *(slides 36)*

$$\boldsymbol{H}^a = \text{Encoder}(I_a), \quad \boldsymbol{H}'^1, \boldsymbol{H}'^2 = \text{Decoder}(\boldsymbol{H}^1, \boldsymbol{H}^2), \quad \hat{\boldsymbol{X}}^{a,1}, C^a = \text{Head}_{3\text{D}}\big([\boldsymbol{H}^a, \boldsymbol{H}'^a]\big).$$

- **Siamese encoder:** one ViT-Large (a vision transformer over image patches) encodes both images with *shared weights* into tokens $\boldsymbol{H}^a$.
- **Two ViT-Base decoders** exchange information through *cross-attention* and output tokens $\boldsymbol{H}'^a$.
- **Heads** regress a pointmap $\hat{\boldsymbol{X}}^{a,1}$ and a per-pixel confidence C^a from the concatenated tokens.
- **MASt3R** (blue on the slide) extends DUSt3R with local descriptors and fast reciprocal matching.

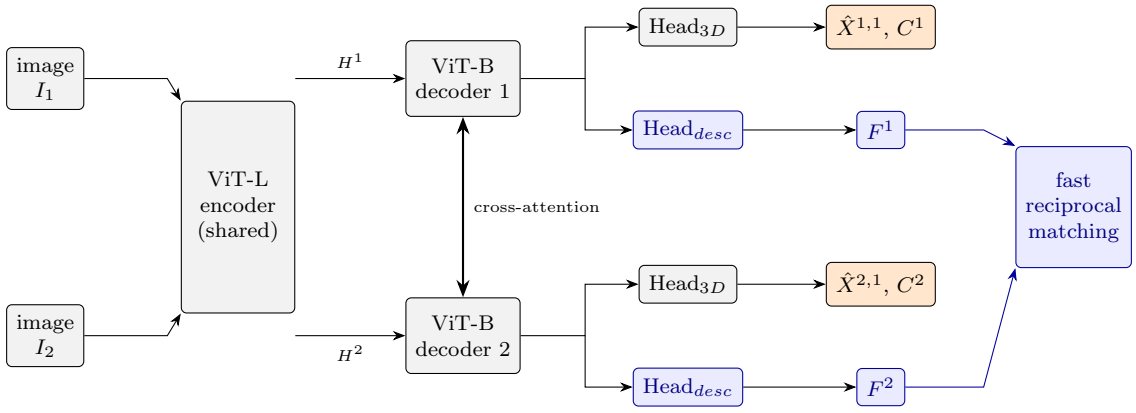


Figure 32: DUSt3R/MASt3R architecture (redrawn from slides 36 and 39; blue = MASt3R additions).

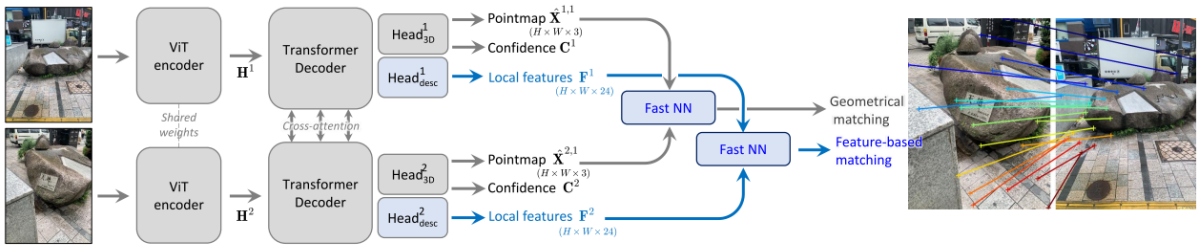


Figure 33: Photos at the left (input images) and right (resulting geometric matching and feature-based fast matching visuals) of the architecture figure

Intuition

The encoder turns each image into a bag of patch tokens independently; the decoders then let the two images "talk" through cross-attention so that each token can relate to the other view. The heads translate tokens into per-pixel 3D coordinates. Crucially, both outputs are in the *same* frame, which is what makes 3D matching possible.

4.5.5 MASt3R Network Output *(slides 37)*

Per image, the network outputs three dense maps: a pointmap $\hat{\boldsymbol{X}} \in \mathbb{R}^{H\times W\times 3}$ (with $\hat{\boldsymbol{X}}[i] \in \mathbb{R}^3$), a confidence $C \in \mathbb{R}^{H\times W}$ (with $C[i] \in \mathbb{R}$) and a local descriptor map $F \in \mathbb{R}^{H\times W\times 24}$ (with $F[i] \in \mathbb{R}^{24}$).

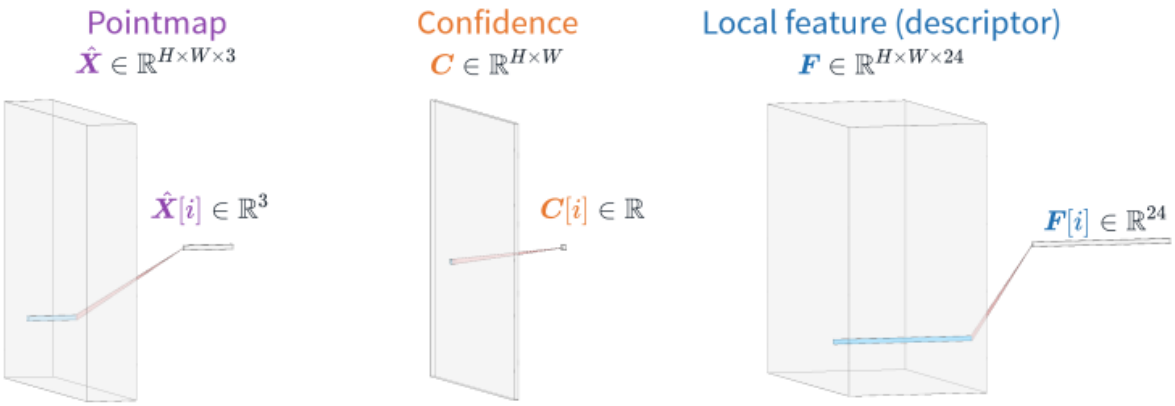


Figure 34: Three 3D-slab sketches: pointmap $\hat{X}$, confidence C and local feature F with a highlighted pixel

4.5.6 Training Objective and Metric Scale *(slides 38)*

Definition 4.5.3 (Confidence-weighted regression loss)**.** With $\rho, \hat{\rho}$ the mean distances of the valid ground-truth and predicted points to the origin,

$$\ell_{\text{regr}}(a,i) = \left\| \tfrac{1}{\hat{\rho}} \hat{\boldsymbol{X}}^{a,1}[i] - \tfrac{1}{\rho} \boldsymbol{X}^{a,1}[i] \right\|, \qquad \mathcal{L}_{\text{conf}} = \sum_{a\in\{1,2\}} \sum_{i\in\mathcal{V}^a} C^a[i]\, \ell_{\text{regr}}(a,i) - \alpha \log C^a[i],$$

$\mathcal{V}^a$ the pixels of I_a with valid ground truth, $\alpha = 0.2$. Dividing by $\rho, \hat{\rho}$ removes the unknown scale.

Proposition 4.5.4 (Optimal confidence)**.** For a fixed error $\ell > 0$, the function $C \mapsto C\ell - \alpha \log C$ on $C > 0$ is minimised at $C^* = \alpha/\ell$, with minimum value $\alpha\big(1 + \log(\ell/\alpha)\big)$.

Proof. The derivative $\ell - \alpha/C$ vanishes at $C = \alpha/\ell$, and the second derivative $\alpha/C^2 > 0$ shows it is a minimum. □

Example 4.5.5. With $\alpha = 0.2$: a pixel with error $\ell = 0.1$ gets $C^* = 2$; one with $\ell = 1$ gets $C^* = 0.2$. (The paper's parametrisation of C may constrain its range; the proposition is the unconstrained picture.)

Intuition

Without the $-\alpha \log C$ term the network could "switch off" every hard pixel by shrinking its confidence towards zero and the loss would vanish. The log term charges a price for low confidence, so the best compromise is a confidence *inversely proportional to the expected error*: it learns to say "I am unsure here" only where it really is (sky, reflections, textureless areas).

Note (from the original paper, not from the slides): to my knowledge, for datasets with metric ground truth MASt3R does not apply this normalisation, which is how it learns metric scale; the slides only state the normalisation and later say the metric scale "comes from a learned prior".

4.5.7 The Matching Head *(slides 39)*

Regressed 3D points are robust to viewpoint changes but imprecise. MASt3R therefore adds a second head per image producing dense local descriptors from the same tokens:

$$F^a = \mathrm{Head}^a_{\mathrm{desc}}\big([\boldsymbol{H}^a, \boldsymbol{H}'^a]\big) \in \mathbb{R}^{H \times W \times 24}, \quad a \in \{1, 2\}.$$

It is a two-layer MLP with a GELU activation; the descriptor $F^a[i] \in \mathbb{R}^{24}$ of each pixel is normalised to unit length. It is trained jointly with the 3D head, so the descriptors stay *grounded* in the pointmaps.

4.5.8 InfoNCE and the Hinge Loss *(slides 40)*

Definition 4.5.6 (Matching loss)**.** With temperature $\tau = 0.07$ and $s_\tau(i, j) = \exp\big(F^1[i]^\top F^2[j]/\tau\big)$,

$$\mathcal{L}_{\mathrm{match}} = -\sum_{(i,j)\in\mathcal{M}} \left(\log \frac{s_\tau(i,j)}{\sum_{i' \in \mathcal{P}^1} s_\tau(i', j)} + \log \frac{s_\tau(i,j)}{\sum_{j' \in \mathcal{P}^2} s_\tau(i, j')} \right).$$

$\mathcal{M}$: ground-truth correspondences (pixel pairs with the same ground-truth 3D point); $\mathcal{P}^1, \mathcal{P}^2$: their pixels in each image. The total loss is $\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{conf}} + \beta \mathcal{L}_{\mathrm{match}}$ with $\beta = 1$.

Proposition 4.5.7 (InfoNCE contains a smooth hinge)**.** If only one positive and one negative candidate are present, each term equals $\log\big(1 + e^{-(u_+ - u_-)/\tau}\big)$ with $u_\pm$ the descriptor dot products. This is a smooth version of the hinge loss of Sec. 4.3 (compare $\tau \log(1 + e^{-\Delta/\tau}) \to \max(0, -\Delta)$ as $\tau \to 0$).

Proof. $-\log \dfrac{e^{u_+/\tau}}{e^{u_+/\tau} + e^{u_-/\tau}} = \log\big(1 + e^{-(u_+ - u_-)/\tau}\big)$. □

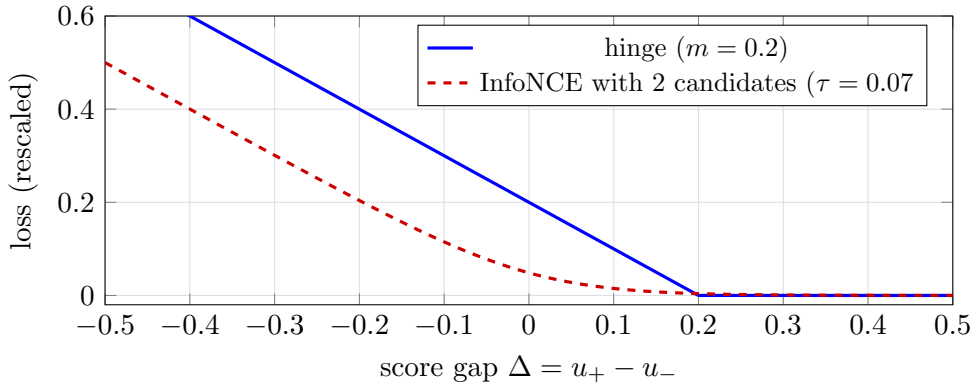


Figure 35: Hinge versus a two-candidate InfoNCE term (new figure; the second is multiplied by τ so the slopes are comparable). InfoNCE never becomes exactly zero: it keeps pushing already-separated pairs apart.

Intuition

The hinge loss compares the match with *one* negative and a fixed margin. InfoNCE is a cross-entropy over *all* candidates: every other matched pixel acts as a negative and only the exact pixel is rewarded. That gives a much richer learning signal per correspondence, and the low temperature makes the descriptor space sharply peaked.

4.5.9 Effect of 3D Grounding *(slides 41)*

Ablation on the Map-free relocalization benchmark (validation set; estimate the metric pose of a query from a single reference image). All rows take depth from the same monocular depth network.

	model, matching on	VCRE AUC ↑	median error ↓	pose AUC ↑
I	DUSt3R, 3D points	0.704	1.10 m, 9.4°	0.344
II	MASt3R, 3D points	0.732	0.94 m, 3.6°	0.409
III	MASt3R, matching loss only, descriptors	0.744	1.10 m, 10.8°	0.382
IV	MASt3R, descriptors	**0.752**	**0.93 m, 3.0°**	**0.435**

VCRE AUC: area under the curve of the reprojection error of virtual 3D points under the estimated pose, up to 90 px. Pose AUC: area under the curve of pose accuracy.

- Descriptors beat matching on 3D points (II vs. IV: 0.732 → 0.752).
- Without the 3D regression loss, the median rotation error grows from 3.0° to 10.8° (III vs. IV).

Intuition

Descriptors give *precise* matches; the 3D loss gives *geometric sanity*. Row III shows that a descriptor trained alone has decent reprojection scores but poor rotation – the matches are locally sharp yet not globally consistent. Training both together gives the best of each.

4.5.10 Reciprocal Matching *(slides 42)*

Definition 4.5.8 (Reciprocal (mutual nearest-neighbour) matches)**.**

$$\hat{\mathcal{M}} = \big\{(i,j) \mid j = \mathrm{NN}_2(F^1[i]) \text{ and } i = \mathrm{NN}_1(F^2[j])\big\}, \qquad \mathrm{NN}_2(F^1[i]) = \arg\min_j \|F^2[j] - F^1[i]\|,$$

and NN_1 is the same from image 2 to image 1.

Lemma 4.5.9. $\hat{\mathcal{M}}$ is a partial one-to-one matching: no pixel of either image appears in more than one pair.

Proof. If $(i,j),(i,j') \in \hat{\mathcal{M}}$ then $j = \mathrm{NN}_2(F^1[i]) = j'$ (nearest neighbour is a function, ignoring ties); symmetrically for a pixel j appearing with two i's. □

Example 4.5.10 (1D toy descriptors)**.** $F^1 = (0,\ 1,\ 5)$ and $F^2 = (0.9,\ 5.2,\ 6)$. Then NN_2 sends $0 \to 0.9$, $1 \to 0.9$, $5 \to 5.2$, while NN_1 sends $0.9 \to 1$, $5.2 \to 5$, $6 \to 5$. Mutual pairs: $(1, 0.9)$ and $(5, 5.2)$. The pair $(0, 0.9)$ is rejected because 0.9 prefers 1.

- It is the left-right consistency test of Sec. 4.2, applied over the whole image instead of one row.
- Without rectification there is no epipolar line: every pixel of image 2 is a candidate, so a naive implementation compares all pixel pairs, $O(W^2H^2)$. For a 512×384 image this is $\approx 3.9 \times 10^{10}$ pairs (illustrative size).

Intuition

It is the same "two witnesses must agree" idea as the left-right test, but without a row to search along. Mutual agreement is a cheap, strong outlier filter; the price is a huge candidate set, which is why MASt3R uses a *fast* matching scheme (the slides name it but do not detail it).

4.5.11 MASt3R Training Details *(slides 43)*

- **Data:** 14 datasets (indoor, outdoor, synthetic, real, object-centric), 10 with metric ground truth (Habitat, ARKitScenes, BlendedMVS, MegaDepth, Static Scenes 3D, ScanNet++, CO3D-v2, Waymo, Map-free, WildRGB-D, Virtual KITTI 2, Unreal4K, TartanAir and an internal dataset).
- **Amount:** 650k pairs per epoch, equally from all datasets, 35 epochs: 650k × 35 = 22.75 million pairs, batch size 64.
- **Images:** 512 px on the long side, five aspect ratios, random crops and colour jitter.
- **Model:** DUSt3R network (ViT-L encoder, ViT-B decoder), initialised from the public DUSt3R checkpoint.
- **Optimisation:** AdamW, learning rate 10^{-4} with 7 warm-up epochs and cosine decay, weight decay 0.05; 4096 ground-truth correspondences per pair for the matching loss.
- Hardware and training time: not reported in the paper.

4.5.12 Results *(slides 44)*

Map-free relocalization, test set	
method	VCRE AUC ↑
LoFTR, monocular depth	0.634
DUSt3R	0.697
MASt3R	**0.933**

DTU multi-view stereo, zero-shot (mm)			
method	Acc. ↓	Comp. ↓	Overall ↓
DUSt3R	2.677	0.805	1.741
MASt3R	**0.403**	**0.344**	**0.374**

DTU: Acc. is the mean distance from the reconstruction to the ground truth (structured light), Comp. the reverse, Overall their mean.

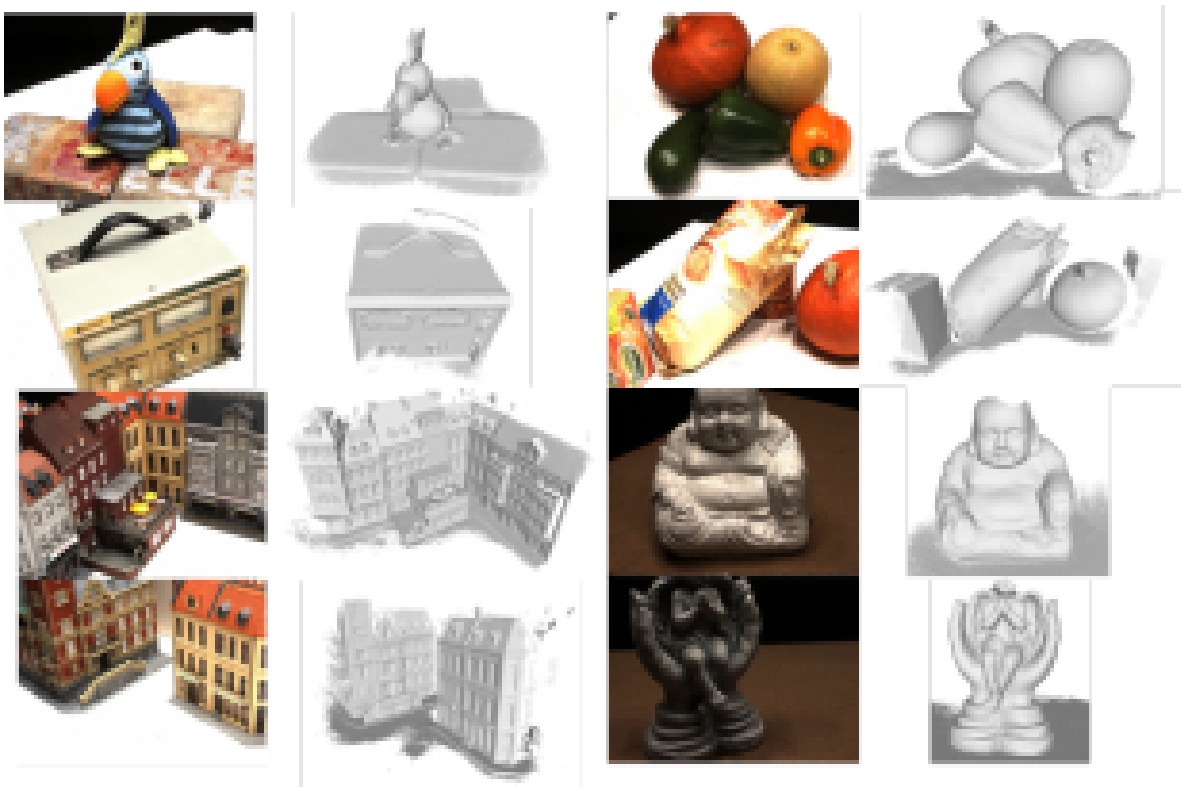


Figure 36: Grid of object photos/reconstructions from DTU

4.5.13 The Pipeline Revisited *(slides 45)*

Step	Calibrated stereo	MASt3R
1. Calibration	measured offline, required	not needed to match; focal estimated; known intrinsics for the best poses
2. Rectification	required	not used
3. Disparity	1D search along a row	reciprocal nearest neighbours over the whole image
4. Outliers	left-right consistency test	reciprocity, the same test in 2D
5. Depth	$z = fb/d$, metric from the measured baseline	pointmap, metric scale from a learned prior
6. 3D model	fusion of depth maps	pointmaps, or triangulation of the matches

Given up: a measured metric scale, full resolution (512-pixel windows), low compute, and more than two images per inference. A calibrated rig still wins for real-time and embedded systems and whenever metric scale must be *measured*.

Intuition

Each step of the classical recipe has a counterpart, but the machinery moves from "geometry we measure" to "geometry the network has learned". You gain freedom (arbitrary cameras, wide baselines, forward motion) and lose guarantees (scale, speed, resolution).

Summary

Comparison of the methods covered

Method	Matching signal	Needs calibration + rectification?	Search / output	Main strength / weakness
Block matching (SSD/ZNCC)	hand-crafted patch cost, WTA	yes	1D along row; disparity	simple; noisy, fronto-parallel assumption, border bleeding
Siamese CNN (Zbontar–LeCun)	learned cost (MLP) or cosine of learned features	yes	1D along row; cost volume + WTA/SGM	far less noise; slow if MLP head; ambiguity remains
DispNet	correlation layer, end-to-end regression	yes (fixed disparity range)	disparity at several scales	fast, uses synthetic pre-training; rectified input only
GC-Net	3D conv on cost volume, soft argmin	yes (fixed disparity range)	sub-pixel disparity	better accuracy; large memory (3D volume), smoothness bias
DUSt3R / MASt3R	pointmaps + 24-d descriptors, reciprocal NN	no (focal estimated)	whole image; pointmap, matches, pose	arbitrary views and wide baselines, scale from a prior; no measured scale, lower resolution, compute heavy

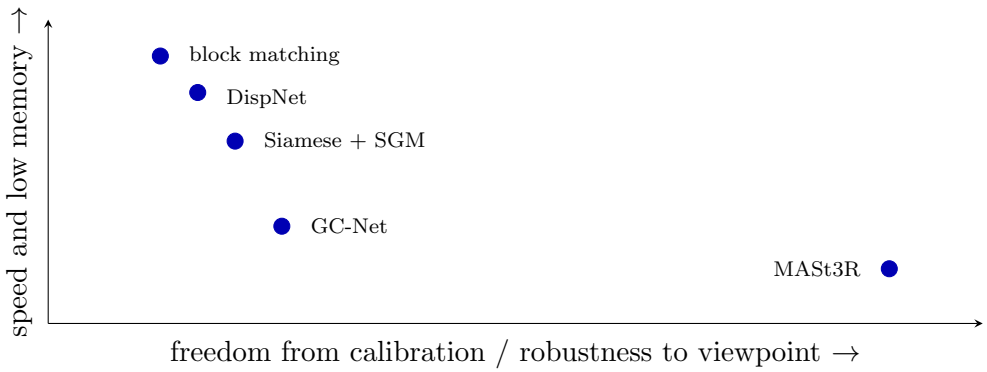


Figure 37: Qualitative placement of the methods on two axes, drawn from the discussion on the slides (calibration/rectification dependence, memory and compute); positions are *not* measured values.

Key take-aways

- **Geometry buys you a 1D search.** With calibration and rectification, matching is along a scanline (a 480× smaller search for VGA) and depth follows from $z = fb/d$.
- **Stereo error is quadratic in depth:** $\delta z \approx z^2 \delta d/(fb)$. A wider baseline helps the arithmetic but makes matching harder.
- **Block matching fails where local similarity fails** (occlusions, slanted surfaces, repetitions, textureless areas); window size trades noise against border bleeding, and the left-right consistency test removes many outliers.
- **Learning replaced the hand-made parts in two steps:** learned matching costs (Siamese nets, hinge loss on triplets), then end-to-end networks (DispNet, GC-Net with cost volumes and soft argmin) – still tied to rectified input and a fixed disparity range.
- **Matching in 3D drops calibration and rectification from the matching step** (DUSt3R/MASt3R: pointmaps, descriptors, reciprocal matching) at the price of measured metric scale, resolution, compute and the number of views; calibrated rigs still win for real-time and embedded, metrically accurate use.